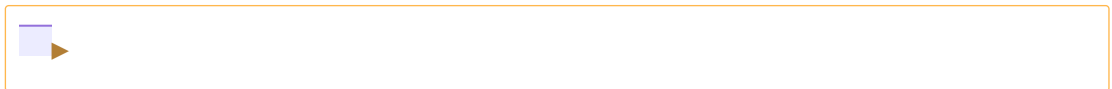


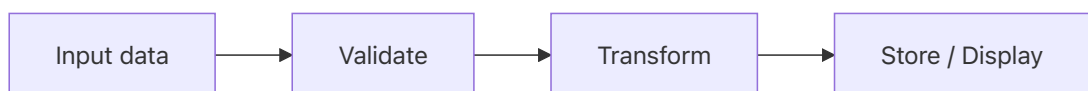
Python execution model

- Python is dynamically typed and interpreted, with most checks happening at runtime.
- Python code can be written in plain text files with the `.py` extension, not only in Jupyter notebooks (or Google Colab notebooks)
- Jupyter notebooks run code cell by cell; `.py` files commonly used to represent complete and reusable programs.
- A `.py` file can be run directly using the Python interpreter (e.g., `python script.py`).
- When a `.py` file is run, Python automatically compiles it before execution.
- The compilation step produces `--bytecode--`, [a low-level representation of the program](#), which is further executed by [Python Virtual Machine](#).

Sequential Logic



Data Flow



```
In [ ] print(2)
```

2

```
In [ ] # print(float(input()))
```

Input and Output

```
In [ ] h = 10
h = 'hello'
print(h)
```

```
2.0 + 2.0
```

```
hello
```

```
Out[... 4.0
```

```
In [... a = True
print(type(a))

a = 1
print(type(a))

# len(a)
# a + ' ' + '2.3'
# # int(a + 2.3)
```

```
<class 'bool'>
```

```
<class 'int'>
```

```
In [... a = 'suspicious'
len(a)
f'something {a}'
name = 'Debjit'
print('Hello Debjit')
# more on fstrings: https://realpython.com/python-f-s
print(f'Hello {name}')
type(a)
```

```
Out[... 10
```

```
Out[... 'something suspicious'
```

```
Hello Debjit
```

```
Hello Debjit
```

```
Out[... str
```

Python Data Types

In Python, every value has a data type. Since Python is **dynamically typed**, you don't need to declare the type of a variable explicitly; the interpreter infers it at runtime.

```
In [... var1 = 2
var2 = 5.0
var3 = False
var4 = "Machine Learning"
```

```
print("Value of var1 var2:", var2, var1) #wrong!
print(f"Value of var1 : {var1}, var2: {var2}")
print("Value of var2 :", var2)
print("Value of var3 :", var3)
print("Value of var4 :", var4)

type(var1), type(var2), type(var3), type(var4)
```

```
Value of var1 var2: 5.0 2
Value of var1 : 2, var2: 5.0
Value of var2 : 5.0
Value of var3 : False
Value of var4 : Machine Learning
```

Out[... (int, float, bool, str)

```
In [... temp = 25
f"The current temperature is {temp}."
'2+3'
for i in range(5):
    print(f'guntur_{i+1}.jpg')
```

Out[... 'The current temperature is 25.'

Out[... '2+3'

```
guntur_1.jpg
guntur_2.jpg
guntur_3.jpg
guntur_4.jpg
guntur_5.jpg
```

Numeric Types

Python supports integers, floating-point numbers, and complex numbers.

```
In [... for i in range(5):
        f'vij_aug2026_{i+1}.jpg'
```

Out[... 'vij_aug2026_1.jpg'

Out[... 'vij_aug2026_2.jpg'

Out[... 'vij_aug2026_3.jpg'

```
Out[... 'vij_aug2026_4.jpg'
```

```
Out[... 'vij_aug2026_5.jpg'
```

```
In [... # Integer (int)
x = 10
print(f"x is {type(x)}")

# Float (float): Numbers with decimals
y = 10
print(f"y is {type(y)}")

# Complex (complex): written with a 'j' for the imaginary part
z = 3 + 5j
print(f"z is {type(z)}")
z.conjugate()
z.__abs__()

x is <class 'int'>
y is <class 'int'>
z is <class 'complex'>
```

```
Out[... (3-5j)
```

```
Out[... 5.830951894845301
```

```
In [... import cmath
cmath.polar(z)
```

```
Out[... (5.830951894845301, 1.0303768265243125)
```

Sequence Types

Sequences allow you to store multiple items in an organized manner.

Strings (str)

Strings are sequences of Unicode characters wrapped in quotes.

```
In [... message = f"Hello, student!"
print(message)

print(f"First character: {message[0]}")      # Indexing
print(f>Last character: {message[-1]}")      # Indexing

# Slicing: access `n` characters from beginning and end
```

```
n = 1
print(f'{message[0:5]}')
print(f"First {n}: {message[:n]}")
print(f"Last {n}: {message[-n:]}")
message
# message[7:]
```

Hello, student!
First character: H
Last character: !
Hello
First 1: H
Last 1: !

Out[... 'Hello, student!'

```
In [...] a = 'srm-ap'
a[4:]
```

Out[... 'ap'

Lists (list)

Lists are **mutable** (changeable) ordered collections that can hold mixed data types.

```
In [...] lst = [1, 1.1, 'a']
lst[1:3]
# for i in lst:
#     print(i)
```

Out[... [1.1, 'a']

```
In [...] fruits = ["apple", "banana", "mango"]
# fruits.insert(1,'a') # inserting
fruits
# id(fruits) # location of the list

print(f'I like to eat {fruits[2]}'.')
# type(fruits[1])
fruits[2][3]
fruits.append("orange")
fruits[2] = "blueberry"
print(f"Modified List: {fruits}")
```

Out[... ['apple', 'banana', 'mango']

I like to eat mango.

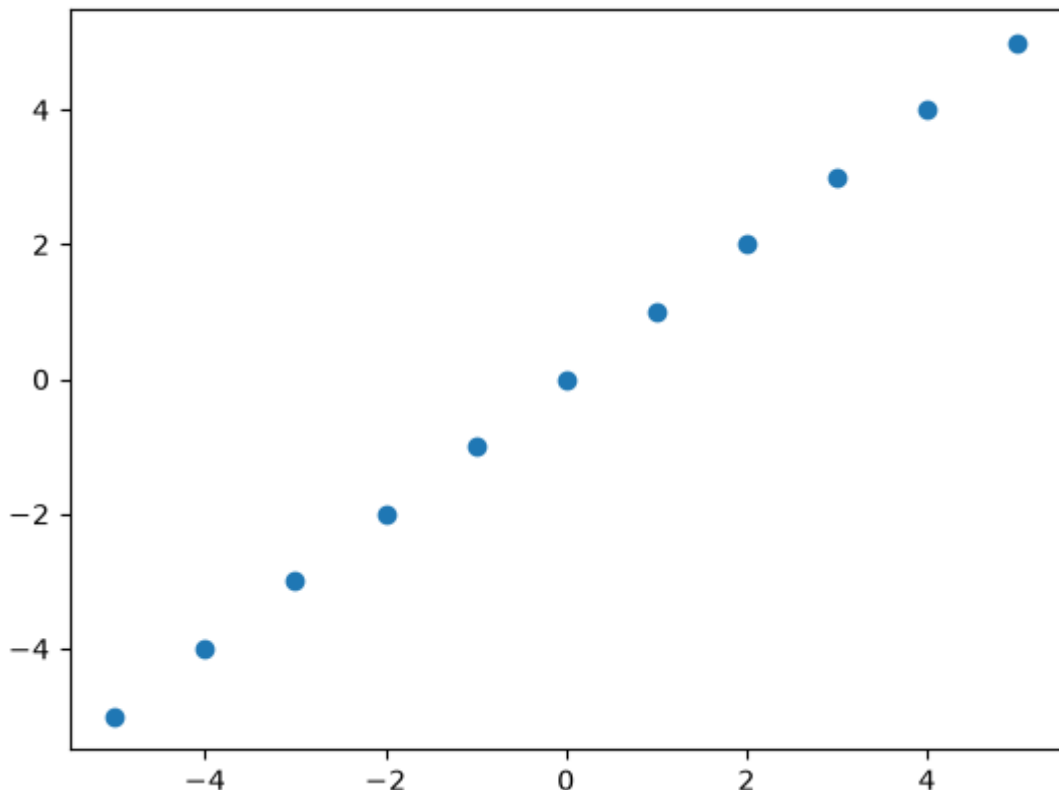
Out[... 'g']

Modified List: ['apple', 'banana', 'blueberry', 'orange']

```
In [... # Example of how you can use a list!
xvals = [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
def parabola(x):
    return x
y = []
for x in xvals:
    y.append(parabola(x))
print(y)
import matplotlib.pyplot as plt
plt.scatter(xvals,y)
```

[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]

Out[... <matplotlib.collections.PathCollection at 0x12a5e2600>



```
In [... # lists can contain heterogeneous data types
```

```
items = [1, "abc", 3.5]
print(type(items[2]))
```

```
<class 'float'>
```

Tuples (tuple)

Tuples are **immutable** ordered collections. Once created, they cannot be changed.

```
In [... coordinates = (15, 20, 30)
coordinates[0:2]
print(f"Tuple: {coordinates}")
# coordinates[0] = 15 # This would raise a TypeError
```

```
Out[... (15, 20)
Tuple: (15, 20, 30)
```

Mapping Type: Dictionary (dict)

Dictionaries store data in **key–value pairs**. They are mutable and indexed by keys.

```
In [... user = {
    "name": "Alice",
    "age": 25,
    "is_member": True
}
print(f"User Name: {user['name']}")

# user['weight']
x = {0: 'name', 1:
    'age'}

print(x[0], x[1])
```

```
User Name: Alice
name age
```

```
In [... d = {5.5: 'ab'}
d[5.5]
# d = [2]
# d[5]
```

```
Out[... 'ab'
```

Set Types (set)

A set is an unordered collection of **unique** items.

```
In [... numbers = {1, 'A', 'A', 'B', 2, 2, 3, 4, 4}
numbers
# print(f"Unique Set: {numbers}") # Removes duplicate
name = 'Arghya'
# name.lower()
print(f'{set(name.lower())}')

from collections import Counter
prt = 'AFDKSDJHFADJADFALKJDLFKJLD'
counter = Counter(prt)
counter
counter[0]
set(prt)
```

```
Out[... {1, 2, 3, 4, 'A', 'B'}
{'g', 'a', 'y', 'r', 'h'}
```

```
Out[... Counter({'D': 6, 'A': 4, 'F': 4, 'J': 4, 'K': 3, 'L'
: 3, 'S': 1, 'H': 1})
```

```
Out[... 0
```

```
Out[... {'A', 'D', 'F', 'H', 'J', 'K', 'L', 'S'}
```

One complete example

```
In [... lst = [1,2,3,4,5]
lst
```

```
Out[... [1, 2, 3, 4, 5]
```

```
In [... # --- LIST SLICING [start:stop:step] ---
nums = [0, 10, 20, 30, 40, 50]

sub = nums[1:4]      # [10, 20, 30] (index 1 to 3)
reverse = nums[::-1] # [50, 40, 30, 20, 10, 0]
every_other = nums[::2] # [0, 20, 40]
print(sub, reverse, every_other)

x = 'naans'
if x == x[::-1]:
    print('it is a palindrome')
else:
```



```
print('not a palindrome')
```

```
[10, 20, 30] [50, 40, 30, 20, 10, 0] [0, 20, 40]  
not a palindrome
```

```
In [... lst = []  
for i in range(1,10,4):  
    lst.append(i)  
print(lst)
```

```
[1, 5, 9]
```

```
In [... # LIST COMPREHENSIONS: Pythonic way to create a list  
  
normal_squares = []  
for x in range(5):  
    normal_squares.append(x**2)  
  
print(normal_squares)  
print(nums)  
# Basic syntax: [expression for item in iterable if condition]  
squares = [x**2 for x in range(5)]           # [0, 1, 4, 9, 16]  
evens = [x for x in nums if x % 2 == 0]      # [0, 20, 40]  
  
print(squares, evens)
```

```
[0, 1, 4, 9, 16]  
[0, 10, 20, 30, 40, 50]  
[0, 1, 4, 9, 16] [0, 10, 20, 30, 40, 50]
```

```
In [... # nums = list(range(50))  
# evens = []  
# for i in nums:  
#     if i%2 ==0:  
#         evens.append(i)  
#     else:  
#         continue  
# evens
```

```
In [... items = ["apple", "banana"]  
print(type(items))  
items.append("cherry")    # Adds to end  
popped = items.pop(1)     # Removes/returns "banana"  
print(items, popped)
```

```
<class 'list'>  
['apple', 'cherry'] banana
```

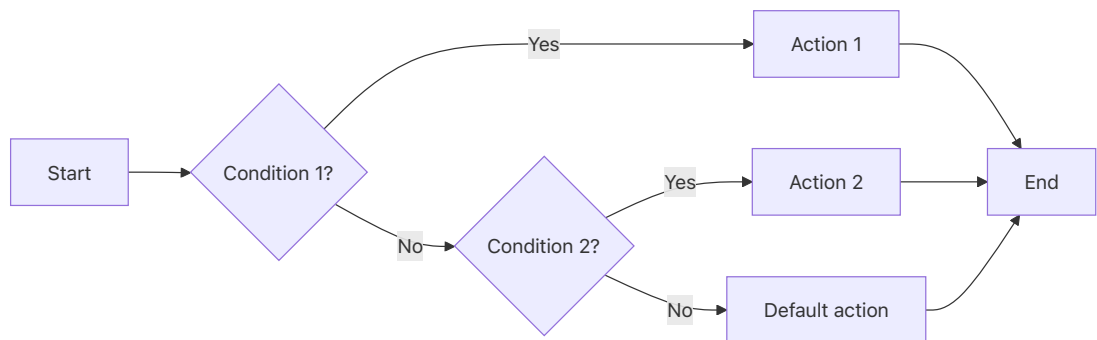
Conditional Statements (if, elif, else)

Conditionals allow to choose between options or take different paths.

Conditional Logic (if / else)



Multiple Conditions (if / else if / else)



```
In [...] temperature = 25

if temperature > 30:
    print("It's a hot day!")
else:
    print("It's a bit chilly.")
```

It's a bit chilly.

```
In [...] temperature = 15

if temperature > 30:
    print("It's a hot day!")
elif temperature > 20:
    print("It's a nice day.")
else:
    print("It's a bit chilly.")
```

It's a bit chilly.

```
In [...] x = 25
```

```

if x >= 50:
    print("A")
elif x >= 25:
    print("B")
else:
    print("C")

```

B

For Loops

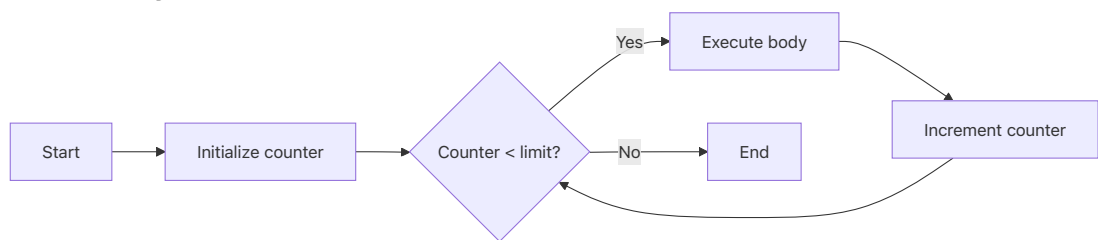
Use a `for` loop when you know how many times you want to run a block of code.

Note that while the underlying model is the same, Python `for` loops works a bit differently: they are controlled by iterators.

```

for x in iterable:
    body

```



```

In [...] t1 = (1,2,3)
          # a = t1[0]
          # b = t1[1]
          a, b, c = t1
          print(a)
          print(b)

```

1
2

```

In [...] lst = ['a','b','c']
          another_lst = ['e','f','g']

          for i in lst:
              print(i)

          # # i = 0
          # # for elem in lst:
          # #     print(elem+another_lst[i])

```

```
# #      i = i+1

# for i, elem in enumerate(lst):
#     print(elem+another_lst[i])
```

a
b
c

```
In [... print(list(range(5)))
[0, 1, 2, 3, 4]
```

```
In [... print("Counting to 5:")
        for i in range(1, 6):
            print(i)

        print("\nIterating through a list:")
        colors = ["Red", "Green", "Blue"]
        for color in colors:
            print(f"Color: {color}")
```

Counting to 5:

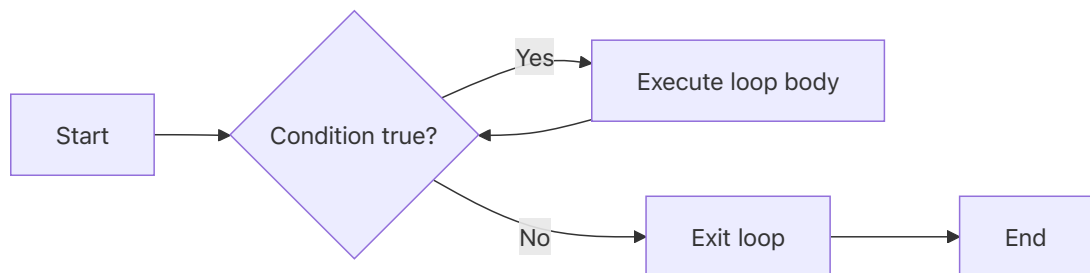
1
2
3
4
5

Iterating through a list:

Color: Red
Color: Green
Color: Blue

While Loops

Use a `while` loop when you want to repeat code as long as a condition is **True**.



```
In [... x = 3

while x > 1:
    print('wow')
    print(x)
    x = x - 1
```

```
wow
3
wow
2
```

```
In [... energy = 3
while energy > 0:
    print(f"Running... Energy left: {energy}")
    energy -= 1
print("Stopped. Need to rest!")
```

```
Running... Energy left: 3
Running... Energy left: 2
Running... Energy left: 1
Stopped. Need to rest!
```

```
In [... # while True:
#     print('wow it never stops')

# while False:
#     print('such sadness')
```

for and while usecases:

- `for` : Use when you want to repeat some action for a known number of times, while iterating over an iterator. Example: processing a **known** number of files in a folder.
- `while` : repeat some action for an unknown number of times, based on some condition.

More: [Check this page from Real Python](#)

Break and Continue

- `break` exits the loop early.
- `continue` skips the current iteration and moves to the next.

```
In [... print("Testing Break (stops at 3):")
for n in range(1, 6):
    if n == 3:
        break
    print(n)

print("\nTesting Continue (skips 3):")
for n in range(1, 6):
    if n == 3:
        continue
    print(n)
```

Testing Break (stops at 3):

1
2

Testing Continue (skips 3):

1
2
4
5

```
In [... for n in range(6):
    # break
    print(n)
    if n == 3:
        break
    # continue
```

0
1
2
3

```
In [... x = 'srm university-ap'
vowels = 'aeiou'
# vowels = ['a', 'e', 'i', 'o', 'u']
for i in x:
    if i in vowels:
        print(i)
    continue
```

u
i
e
i
a

note: pass and continue are not same in python! more here: <https://stackoverflow.com/questions/9483979/is-there-a-difference-between-pass-and-continue-in-a-for-loop-in-python>

```
In [... # one example why all these is going to be useful
filenames = []
for i in range(1,3):
    filenames.append(f'excel{i}')

import pandas as pd
for filename in filenames:
    foo = pd.read_excel(f'{filename}.xlsx')
    foo['product'] = foo['A']*foo['B']
    foo.to_excel(f'updated_{filename}.xlsx', index=None)
```

```
In [... filenames = []
for temp in range(1,3):
    if (temp>1):
        filenames.append(f'excel{temp}.xlsx')
print(filenames)
for filename in filenames:
    _ = pd.read_excel(filename)
    print(_)
# print(filenames)
[f'excel{i}.xlsx' for i in range(1,20,2) if i%2==1]
```



```
['excel2.xlsx']
```

```
    A    B
```

```
0    4   11
```

```
1    5    2
```

```
2    6    3
```

```
Out[... ['excel1.xlsx',  
        'excel3.xlsx',  
        'excel5.xlsx',  
        'excel7.xlsx',  
        'excel9.xlsx',  
        'excel11.xlsx',  
        'excel13.xlsx',  
        'excel15.xlsx',  
        'excel17.xlsx',  
        'excel19.xlsx']
```

```
In [...
```

Functions

- A function is a reusable block of code that performs a specific task
- A function is identified by its name (for example, `sqrt`)
- Parentheses () indicate that the function is being called
- Values inside the parentheses are called arguments, which are passed to the function as input
- The function executes its internal code using the given arguments
- The function computes a result, which is returned to the caller
- The returned value can be printed, stored in a variable, or used in an expression
- The core of the function code block should be indented by 4 spaces.

wrong code

```
def sum(a,b):  
return a+b
```

correct code

```
def sum(a,b):  
    return a+b
```

Reference: <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

Function Call and Return



In [...]

```
def sum(x, y):  
    # print(x)  
    print(x+y)  
    # return x+y
```

```
sum(1,2)
# print(a)
```

3

```
In [...]: def foo(a):
            sum = 0
            for i in a:
                sum = sum + i
            return sum/len(a)
foo([2,3,4])
```

Out[...]: 3.0

```
In [...]: def addElements(a, b):
            return a + b

addElements(2, 3)
addElements(2, 3.1)
addElements(2.3, 4.5)
addElements("python", " workshop")
# addElements(2) # error!
```

Out[...]: 5

Out[...]: 5.1

Out[...]: 6.8

Out[...]: 'python workshop'

```
In [...]: # when the order is important!
```

```
def subtract(a,b):
    return a-b
```

```
subtract(2,3)
subtract(3,2)
subtract(b=3,a=2)
```

Out[...]: -1

Out[...]: 1

Out[...]: -1

```
In [... def greet(name, first='Hello,'):  
# def greet(name, first):  
    """  
    docstring: write something useful!  
    """  
    # print(f'{first} {name}!')  
    return f'{first} {name}!'
```

print(greet('Sindhu', 'Hello,'))

Hello, Sindhu!

```
In [... import numpy as np  
np.arange(0,10,1)
```

```
Out[... array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [... def multdiv(x, y, mult=True):  
    """  
    this is a horrible function, you shouldn't use th  
    """  
    if mult:  
        result = x * y  
    else:  
        result = x / y  
    return result
```

multdiv(2, 2)
multdiv(2, 2, True)
multdiv(2, 2, False)

```
Out[... 4
```

```
Out[... 4
```

```
Out[... 1.0
```

```
In [... def repeat(s, exclaim=False):  
    "A completely useless function, but hey it has a  
    result = s * 3  
    if exclaim:  
        result += "!!!"  
    return result
```

repeat("Yay, ")

```
# repeat(3)

# repeat('Yay, ', False)
repeat('Woo ', True)
```

```
Out[... 'Yay, Yay, Yay, '
```

```
Out[... 'Woo Woo Woo !!!'
```

```
In [... # default values for parameters
def addElements(a, b=3):
    return a + b
```

```
addElements(2)
addElements(2, 5)
```

```
Out[... 5
```

```
Out[... 7
```

A complete example

with some advanced bits..

```
In [... def foo(a, b=4):
        c = 3
        return 3+c+a
foo(4,0)
# print(c)
```

```
Out[... 10
```

```
In [... # --- GLOBAL SCOPE ---
x = "Global Variable"

def example_function(required_param, default_param="D
    """
    DOCSTRING: This is where you explain what the fun
    It takes two parameters and returns a formatted s
    """
```

```

# --- LOCAL SCOPE ---
# Variables defined here only exist inside this f
internal_val = "Local Variable"

result = f"{required_param} | {default_param} | {"
return result
# print(internal_val)

```

```

In [... # 1. Calling with Positional Arguments
print(example_function("Hello", "Custom Value"))

```

Hello | Custom Value | Local Variable

```

In [... # 2. Calling with Keyword Arguments (Order doesn't ma
print(example_function(default_param="Keyword", requi

```

World | Keyword | Local Variable

```

In [... # 3. Using Default Argument
print(example_function("Just Required"))

```

Just Required | Default Value | Local Variable

```

In [... # 4. Scope demonstration
print(x) # Works (Global)
# print(internal_val) # NameError (Local scope only)

```

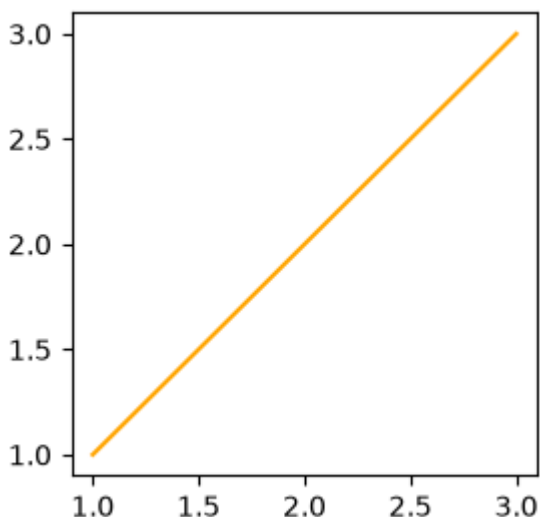
Global Variable

```

In [... import matplotlib.pyplot as plt
fig,ax= plt.subplots(figsize=(3,3))
ax.plot([1,2,3],[1,2,3],color='orange')

```

Out[... [<matplotlib.lines.Line2D at 0x12ffa1e20>]



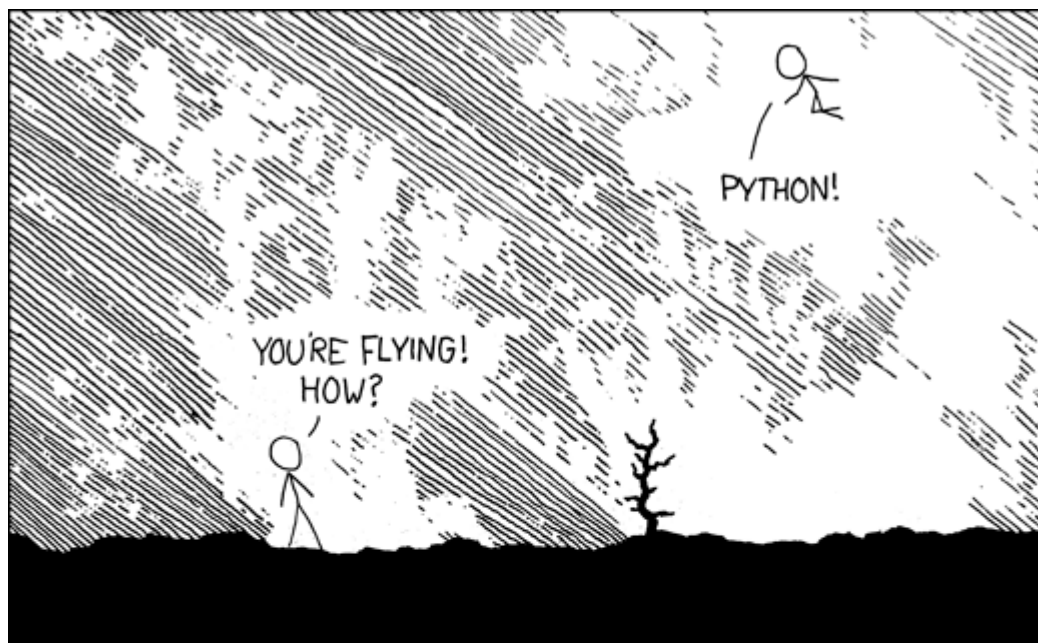
```
In [... # 5. Accessing Docstring
print(example_function.__doc__)
```

DOCSTRING: This is where you explain what the function does. "

It takes two parameters and returns a formatted string.

Modules and Packages and `__main__`

Python files are modules. When run directly, **name** equals **'main'**. See `example_import.py` and `example_main.py`. For more details check [Python's documentation](#).



Source: XKCD

In the following code block:

- `import math` loads the standard Python math module, which provides mathematical functions and constants
- `math.sqrt(16)` calls the `sqrt` (square root) function from the module with input value `16` ; the result is `4.0`
- The result is a float because `math.sqrt` always returns a floating-point value
- Without `import math` , `sqrt` is not available

Reference: <https://docs.python.org/3/library/math.html>

```
In [... import math

math.sqrt(3.498572394)
```

```
Out[...] 1.8704471107197873
```

Ways to use functions/constants from a module

```
In [... # import a single function
from random import sample
sample("Luke", 2)
```

```
Out[...] ['L', 'u']
```

```
In [... # import without an alias. here you need to mention the module
import numpy
print(numpy.absolute(-2))
print(numpy.e)
```

```
2
2.718281828459045
```

```
In [... # import (with an alias). most common use. the alias is used
import numpy as np
print(np.absolute(-2))
```

```
2
```

```
In [... # import the whole module without any alias. this all
```



```
# from numpy import *
import math
import numpy as np
print(math.e)
print(np.e==math.e)
```

2.718281828459045

True

In [... *# more on namespace <https://realpython.com/python-namespaces/>*

```
x = 3

def square(y):
    """
    this function squares a variable
    """
    y = 2
    return y**2

# y # will return an error

import sum_module
sum_module.sum(2,3)
```

Out[...] 5

In [... *# why should we use loops and functions?*

```
10**2+2
20**2+2
30**2+2
```

Out[...] 102

Out[...] 402

Out[...] 902

In [...

```
lst = [10,20,30]

# lst= list(range(10000))

def func(x):
    return x**2+2

for i in lst:
```

```
print(func(i))
```

102

402

902

```
In [... # function use-case
import matplotlib.pyplot as plt
def plot_wave(ax, frequency):
    x = np.linspace(0, 2*np.pi, 100)
    y = np.sin(frequency * x)
    ax.plot(x, y)
    ax.set_title(f"$f = {frequency}$")

fig, axes = plt.subplots(1, 3, figsize=(6, 2), sharey

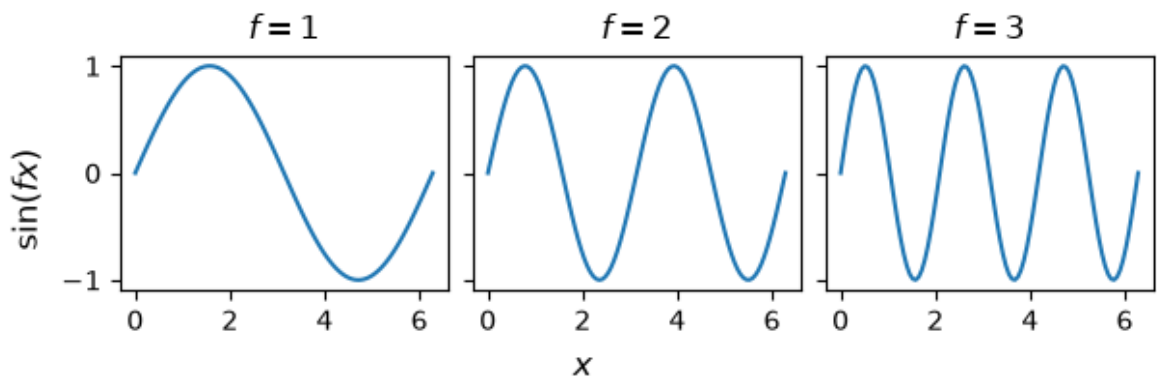
fig.supylabel(r"$\sin (fx)$")
fig.supxlabel(r'$x$')

for i in range(0,3):
    plot_wave(axes[i], i+1)

# for i, elem in enumerate(axes):
#     plot_wave(elem, i+1)
```

Out[... Text(0.02, 0.5, '\$\sin (fx)\$')

Out[... Text(0.5, 0.01, '\$x\$')



```
In [... lst1 = ('n1', 'n2')
```

```
a, b = lst1
```

```
a
```

```
b
```

Out[... 'n1'

Out [... 'n2'

Python's Object Model and Mutability

In Python, **what a variable is** and **what it points to** both matters.

1. Everything is an Object

In Python, everything (integers, strings, functions, even types themselves) is an **object**.

An object has:

1. An **Identity** (its address in memory).
2. A **Type** (e.g., integer, list).
3. A **Value** (the data it holds).

The object model is at the **core of how Python works**. If you're curious, look up chapter 6 and 7 of Ramlho's book. This is not required for this course, but life is boring if you only read what's in the syllabus.

Ramalho L. 2022. Fluent Python: Clear, Concise, and Effective Programming, Second Edition. Shroff/O'Reilly. 1012 p.

```
In [...] # x = [1, 2, 3]
x = 1

print(f"Identity (Memory Address): {id(x)}")
print(f"Type: {type(x)}")
print(f"Value: {x}")
```

```
Identity (Memory Address): 4369722280
Type: <class 'int'>
Value: 1
```

2. Mutability: Mutable vs Immutable

Type	Category	Behavior
Immutable	int, float, str, tuple	Cannot be changed after creation. Changes create a new object.

Type	Category	Behavior
Mutable	list, dict, set	Can be changed in-place without changing the identity.

Why Lists behave differently from Strings/Tuples

When you modify a list, you are changing the contents of that specific memory address. When you "modify" a string, Python actually creates a brand new string and reassigns your variable name to it.

```
In [... # Immutable Example: Strings
s = "Hello"
old_id = id(s)
s = s + " World"
print(old_id, id(s))
print(f"String ID changed: {old_id != id(s)}") # True

# Mutable Example: Lists
l = [1, 2]
old_id = id(l)
l.append(3)
print(old_id, id(l))
print(f"List ID changed: {old_id != id(l)}") # False

4725503728 4725992368
String ID changed: True
4725992192 4725992192
List ID changed: False
```

Functions vs Methods

- **Functions:** Standalone blocks of code called by name (e.g., `print(x)` or `len(x)`).
- **Methods:** Functions that "belong" to an object and are called using dot notation (e.g., `list.append()` or `string.upper()`).

```
In [... my_list = [3, 1, 2]
print(type(my_list))
# len() is a function
```

```
print(len(my_list))

# .sort() is a method belonging to the list object
sorted(my_list)
my_list.sort()
print(my_list)

str_example = 'vamsi Is a good person'
str_example.lower()
str_example.count('v')
```

```
<class 'list'>
```

```
3
```

```
Out[... [1, 2, 3]
```

```
[1, 2, 3]
```

```
Out[... 'vamsi is a good person'
```

```
Out[... 1
```

```
In [... # a quick example of how you can use python methods
# here we want to compute the percentage of different
```

```
insulin = 'MALWMRLLPLLALLLWGPDPAAAFVNQHLCGSHLVEALYLIV
```

```
amino_acids_1_letter = [
    "A", # Alanine
    "R", # Arginine
    "N", # Asparagine
    "D", # Aspartic acid
    "C", # Cysteine
    "E", # Glutamic acid
    "Q", # Glutamine
    "G", # Glycine
    "H", # Histidine
    "I", # Isoleucine
    "L", # Leucine
    "K", # Lysine
    "M", # Methionine
    "F", # Phenylalanine
    "P", # Proline
    "S", # Serine
    "T", # Threonine
    "W", # Tryptophan
    "Y", # Tyrosine
```

```

    "V"    # Valine
]

len_insulin = len(insulin)
print(len_insulin)

percentage_count = {}

for amino_acid in amino_acids_1_letter:
    # print(amino_acid)
    percentage_count[amino_acid] = (insulin.count(ami
    # break
# percentage_count
percentage_count.keys()
percentage_count.values()
import matplotlib.pyplot as plt
plt.bar(percentage_count.keys(),percentage_count.valu

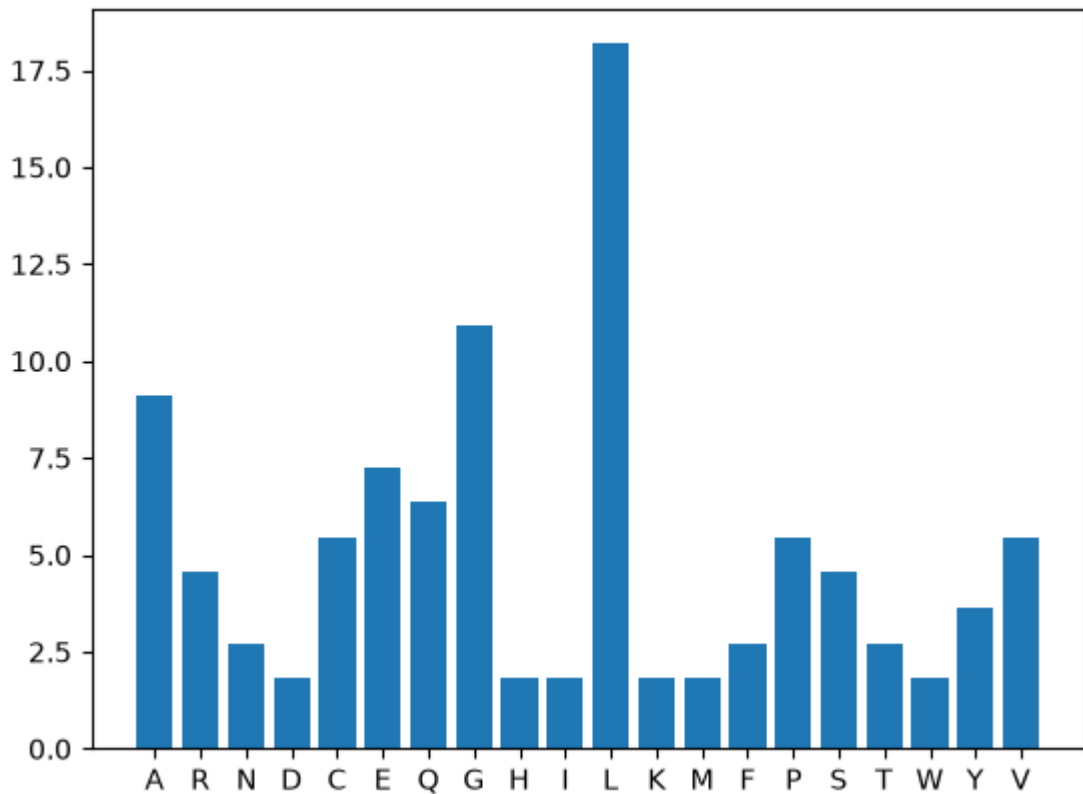
```

110

Out[...] dict_keys(['A', 'R', 'N', 'D', 'C', 'E', 'Q', 'G', 'H', 'I', 'L', 'K', 'M', 'F', 'P', 'S', 'T', 'W', 'Y', 'V'])

Out[...] dict_values([9.090909090909092, 4.545454545454546, 2.727272727272727, 1.8181818181818181, 5.454545454545454, 7.2727272727272725, 6.363636363636363, 10.909090909090908, 1.8181818181818181, 1.8181818181818181, 18.181818181818183, 1.8181818181818181, 1.8181818181818181, 2.727272727272727, 5.454545454545454, 4.545454545454546, 2.727272727272727, 1.8181818181818181, 3.6363636363636362, 5.454545454545454])

Out[...] <BarContainer object of 20 artists>



Reading material:

- Jake Vanderplus has written why Python has slow and what connections it has to do with Python's objects: <https://jakevdp.github.io/blog/2014/05/09/why-python-is-slow/>
- We will not discuss classes and object-oriented programming in this course. If you want to know about it, I recommend Realpython's tutorial: <https://realpython.com/python-classes/>

Python Exceptions and Debugging

Python provides a way to handle these errors (Exceptions) and tools to find their causes (Debugging).

Reference: <https://realpython.com/python-exceptions/>

The Exception Handling Block

Python uses a `try-except` block to manage errors without crashing the program.

- `try` : The code that might cause an error.
- `except` : The code that runs if an error occurs.
- `else` : Runs ONLY if no errors occurred in the `try` block.
- `finally` : Runs no matter what (useful for closing files or cleanup).

In [...]

```
num = 'a'
100/num
```


TypeError

Traceback

(most recent call last)

Cell In[9], line 2

```
1 num = 'a'
----> 2 100/num
```

TypeError: unsupported operand type(s) for /: 'int' and 'str'

In [...]

```
a = 10
try:
    num = 'a'
    # num = float(input("Enter a number to divide 100"))
    # type(num)
    result = 100 / num
except TypeError:
    print("unsupported operand type(s) for /: 'int' and 'str'")
```

```

except ValueError:
    print("Error: That wasn't a valid number!")
except ZeroDivisionError:
    print("Error: You cannot divide by zero!")
else:
    print(f"Success! The result is {result}")
finally:
    print("Execution complete. Cleaning up resources.

```

unsupported operand type(s) for /: 'int' and 'str'
 Execution complete. Cleaning up resources...

In [...] int(20.0)

Out[...] 20

```

In [... ] # num = float(input())
num = float(input())
if type(num) == str:
    print('you entered a string!')
else:
    print(100/num)

```

10.0

```

In [... ] # a = 2
# 2 + a
# '2' + 'a'

lst = [1,2,3]
for i in range(5):
    print(lst[i]) #index error

```

1
 2
 3

```
-----  
-----  
IndexError                                Traceback  
(most recent call last)  
Cell In[15], line 7  
      3 # '2' + 'a'  
      4  
      5 lst = [1,2,3]  
      6 for i in range(5):  
----> 7     print(lst[i]) #index error
```

IndexError: list index out of range

Common Python Exceptions

Recognizing these common errors helps you debug faster:

Exception	Cause
TypeError	Operation applied to an object of inappropriate type (e.g., 'a' + 5).
ValueError	Argument has right type but inappropriate value (e.g., int('hello')).
IndexError	Trying to access a list index that doesn't exist.
KeyError	Trying to access a dictionary key that doesn't exist.

```
In [... d = {'a': 1, 'b':2}
d['a']
d['c']
```

```
Out[... 1
```

```
-----
-----
KeyError                                Traceback
(most recent call last)
Cell In[16], line 3
      1 d = {'a': 1, 'b':2}
      2 d['a']
----> 3 d['c']
```

KeyError: 'c'

Tracebacks

A Traceback is the report Python gives you when code fails. Read it from bottom to top:

1. The last line tells you the error type and a brief description.
2. The lines above show exactly where the error happened (File name and Line number).

Debugging Basics

The "Print" Method

The simplest way to debug is to print variable states at different stages to see where they deviate from your expectations.

Inspection Tools

- `type(obj)` : Check what kind of data you are working with.
- `len(obj)` : Check the size of a collection to avoid `IndexError`.

```
In [... def calculate_average(data):
        print(f"DEBUG: data type is {type(data)}, length
        total = sum(data)
```

```
    return total / len(data)
```

```
my_data = [10, 20, 30]  
my_data  
print(calculate_average(my_data))
```

```
Out[...  [10, 20, 30]
```

```
DEBUG: data type is <class 'list'>, length is 3  
20.0
```

Advanced tools: pdb and IDEs (breakpoints).

Typically physicists don't use them; we are happy with our own print debugging. :P But look them up.

Class

- A template for creating objects that bundles data (attributes) and behavior (methods) together.
- define as `class ClassName:`
- the `__init__` method sets up the initial attributes when an object is created
- Each object made from the class is called an instance
- `self` refers to that specific instance inside the class's methods, letting each object keep its own data while sharing the same behavior.

```
In [... s = 'string'
s.capitalize()
a = [1,3,4]
print(type(a))
```

```
Out[... 'String'
<class 'list'>
```

```
In [... class Person:
    def __init__(self, a):
        self.name = a

    def find_name_length(self):
        print(f'The length of {self.name} is {len(self.name)}')

p1 = Person('debjit')
print(p1.name)
p1.find_name_length()
```

```
debjit
The length of debjit is 6
```

```
In [... class Customer:
    def __init__(self, name, age, current_balance=0):
        self.name = name
        self.age = age
        self.current_balance = current_balance
```

```
def change_balance(self, x):  
    self.current_balance = self.current_balance +
```

```
In [... person1 = Customer("Alice", 25)  
  
# person1.__dir__()  
person1.name  
person1.age  
person1.current_balance  
deposit = -10 # real banks will do something more dec  
person1.change_balance(deposit)  
person1.current_balance
```

```
Out[... 'Alice'
```

```
Out[... 25
```

```
Out[... 0
```

```
Out[... -10
```

```
In [... person2 = Customer("Bob", 20, 56)  
# type(person2)  
person2.name  
person2.age  
person2.current_balance  
deposit = 10 # real banks will do something more dece  
person2.change_balance(deposit)  
person2.current_balance
```

```
Out[... 'Bob'
```

```
Out[... 20
```

```
Out[... 56
```

```
Out[... 66
```

```
In [... class Student:  
    def __init__(self, name, grade):  
        self.name = name  
        self.grade = grade  
  
    def is_passing(self):  
        return self.grade >= 50
```

```
In [... student = Student("John", 5)
```

```
print(student.name)  
print(student.grade)  
print(student.is_passing())
```

John

5

False

Reading and Writing Files in Python

File handling is a fundamental skill. Python makes this easy with the built-in `open()` function and the `with` statement.

1. The `with` Statement (Context Managers)

In older Python code, you might see files opened and closed manually. However, if an error occurs before `.close()` is reached, the file stays open in memory.

The **`with` statement** is a **Context Manager**. It automatically closes the file for you as soon as the code block finishes, even if an error occurs.

2. Writing to a File

Common modes for `open()` :

- `'w'` : **Write** (Overwrites the file if it exists, or creates a new one).
- `'a'` : **Append** (Adds text to the end of the existing file).
- `'r'` : **Read** (Default mode).

```
In [... # Writing to a file
with open("example.txt", "w") as file:
    file.write("Hello World!\nThis file was created u
# file.write("")
```

```
with open("example.txt", 'a') as file:
    file.write('\nThis text was appended later.')
# print("File written successfully!")
```

```
Out[... 73
```

```
Out[... 30
```

```
In [... # csv files are plain text files that stores values s
# this is a widely used format and it's especially us
```

```
# generally, I would advise to use pandas for handling
with open('table.csv', 'w') as file:
    file.write('Number,Square\n')
    for i in range(1,11):
        file.write(f'{i},{i**2}\n')
```

Out[... 14

Out[... 4

Out[... 4

Out[... 4

Out[... 5

Out[... 5

Out[... 5

Out[... 5

Out[... 5

Out[... 5

Out[... 7

3. Reading from a File

You can read the entire content at once, or iterate line by line (which is better for large files).

```
In [... # # Reading the entire file
# with open("table.csv", "r") as file:
#     content = file.read()
#     # print(f"Full Content of {file.name}:")
#     print(content)

# Reading line by line (Memory efficient)
with open("table.csv", "r") as file:
    print(f"Line by Line content of {file.name}:")
    i = 0
    for line in file:
        print(f"{line.strip()}") # .strip() removes t
```

Line by Line content of table.csv:

Number,Square

1,1

2,4

3,9

4,16

5,25

6,36

7,49

8,64

9,81

10,100

In [...]

```
foo = list('abcd')
for i,letter in enumerate('alsdfjld'): # string is a
    print(i,letter)
```

0 a

1 l

2 s

3 d

4 f

5 j

6 l

7 k

8 d

Summary of file modes

Mode	Description	File Pointer Position
'r'	Read (default)	Start of file
'w'	Write (overwrites)	Start of file
'a'	Append (adds to end)	End of file
'r+'	Read and Write	Start of file

In [...]

```
with open('num.txt', 'w') as file:
    for i in range(5):
        file.write(f'{i}\n')
```

Out[... 2

Out[... 2

Out[... 2

Out[... 2

Out[... 2

```
In [... import numpy as np
import matplotlib.pyplot as plt
import timeit
```

NumPy

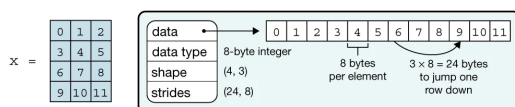
- Efficient and powerful library of arrays
- NumPy arrays are more efficient than Python lists as they are stored in a contiguous block of memory.
- Many functions are available for mathematical operations, linear algebra, and so on.

Resources:

- Beginner: [Numpy official tutorial](#)
- [Official Numpy Quickstart](#)
- [A visual guide to Numpy](#)
- Somewhat advanced: [Nicolas P. Rougier's tutorial](#)
- [M Scott Shell's notes](#)
- Harris, C. R. et al. Array Programming with NumPy. Nature 2020, 585 (7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>.

A neat summary of core numpy methods

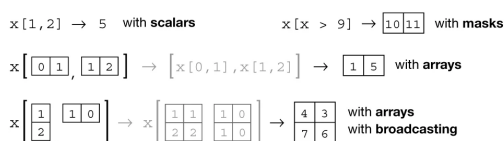
a Data structure



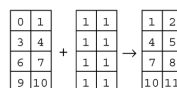
b Indexing (view)



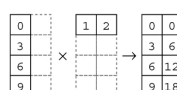
c Indexing (copy)



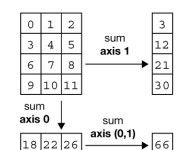
d Vectorization



e Broadcasting



f Reduction



g Example

```
In [1]: import numpy as np
In [2]: x = np.arange(12)
In [3]: x = x.reshape(4, 3)

In [4]: x
Out[4]:
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])

In [5]: np.mean(x, axis=0)
Out[5]: array([4.5, 5.5, 6.5])

In [6]: x = x - np.mean(x, axis=0)

In [7]: x
Out[7]:
array([[ -4.5, -4.5, -4.5],
       [ -1.5, -1.5, -1.5],
       [  1.5,  1.5,  1.5],
       [  4.5,  4.5,  4.5]])
```

Source: Harris et al., Nature (2020). See above

Defining arrays

```
In [... # 1D python list
foo = [1, 2, 3, 4, 5]

# a use case: finding mean of a list of numbers
# using only python
sum=0
for i in foo:
    sum = sum+i
print(sum/5)

# using numpy
arr1 = np.array(foo)
# print(arr1,arr1[2])
print(f'Mean = {arr1.mean()}, sum = {arr1.sum()}')

3.0
Mean = 3.0, sum = 15
```

```
In [... # 2D array
arr2 = np.array([[1, 2, 3], [4, 5, 6]])

print(arr2)
print(arr2[1,2])

[[1 2 3]
 [4 5 6]]
6
```

Array attributes

```
In [... print(arr2)
print(arr2.shape) #shape of the array
print(arr2.dtype) # data type
print(arr2.size) # size of the array
print(arr2.ndim) # number of dimensions

[[1 2 3]
 [4 5 6]]
(2, 3)
int64
6
2
```

```
In [... # 2d arrays in python
a = []
for i in range(2):
    b = []
    for j in range(2):
        b.append(j)
    a.append(b)
print(a)

# numpy
a = np.tile(np.arange(2), (2, 1))
print(a)

[[0, 1], [0, 1]]
[[0 1]
 [0 1]]
```

Initializing arrays

```
In [... zeros_arr = np.zeros((4, 4))
zeros_arr
```

```
Out[...] array([[0., 0., 0., 0.],
               [0., 0., 0., 0.],
               [0., 0., 0., 0.],
               [0., 0., 0., 0.]])
```

```
In [... ones_arr = np.ones((2, 4))
ones_arr
```

```
Out[...] array([[1., 1., 1., 1.],
               [1., 1., 1., 1.]])
```

```
In [... full_arr = np.full((3,2), 'adsfljd')
full_arr
```

```
Out[...] array([[ 'adsfljd', 'adsfljd'],
               [ 'adsfljd', 'adsfljd'],
               [ 'adsfljd', 'adsfljd']], dtype='<U7')
```

```
In [... identity_matrix = np.eye(3)
identity_matrix
```

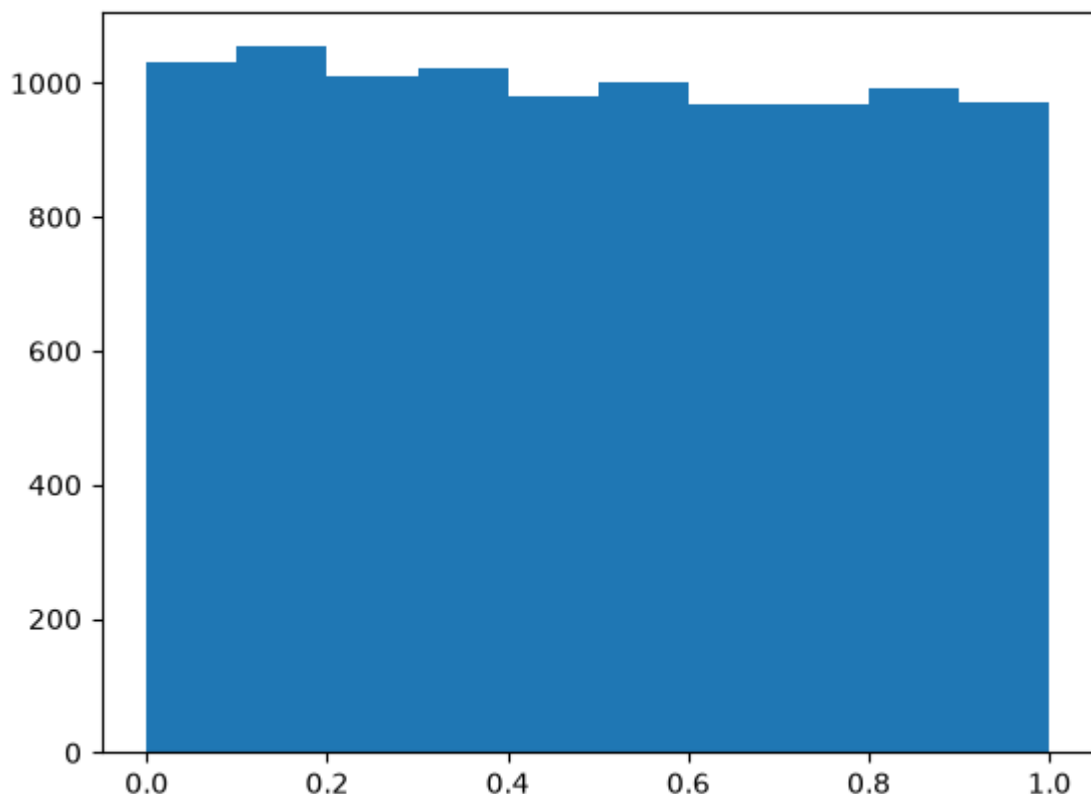
```
Out[...] array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

```
In [... # Array with random values
# np.random.seed(4)
random_arr = np.random.random((10000))

plt.hist(random_arr)
random_arr
```

```
Out[... (array([1031., 1054., 1009., 1023., 981., 1002., 9
69., 969., 992.,
          970.]),
        array([6.36604711e-05, 1.00049200e-01, 2.00034739
e-01, 3.00020278e-01,
          4.00005817e-01, 4.99991356e-01, 5.99976895
e-01, 6.99962434e-01,
          7.99947973e-01, 8.99933512e-01, 9.99919052
e-01])),
        <BarContainer object of 10 artists>)
```

```
Out[... array([0.01442363, 0.15595053, 0.27229018, ..., 0.70
038508, 0.11996656,
          0.20053322], shape=(10000,))
```



Accessing elements

```
In [... arr = np.array([10, 20, 30, 40, 50])
```



```
arr[0] # first element
arr[-1] # last element
```

```
Out[... np.int64(10)
```

```
Out[... np.int64(50)
```

```
In [... # Slicing
arr[1:3]
arr[::2] # Every second element
arr[:2]
```

```
Out[... array([20, 30])
```

```
Out[... array([10, 30, 50])
```

```
Out[... array([10, 20])
```

```
In [... # Multi-dimensional arrays
arr2 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr2)
print(arr2[1, 2])
print(arr2[:, 2:]) # third column as a column vector
print(arr2[:, 2:].shape) # third column as a column v

[[1 2 3]
 [4 5 6]
 [7 8 9]]
6
[[3]
 [6]
 [9]]
(3, 1)
```

Boolean masking

Select elements using conditions.

```
In [... x = np.array([1, -2, 3, -4])
print(x[x > 0])

[1 3]
```

Aggregations with axis

Understand operations along dimensions.

```
In [... arr = np.array([[1,2,3],[4,5,6]])
print(arr)
arr.mean(axis=0)
arr.mean(axis=1)
```

```
[[1 2 3]
 [4 5 6]]
```

```
Out[... array([2.5, 3.5, 4.5])
```

```
Out[... array([2., 5.])
```

Views vs copies

Slicing usually returns a view.

```
In [... a = np.array([1,2,3,4])
# b=a.copy()
b = a[1:3].copy()
id(a)
id(b)

print(b)
b[0] = 99
b
a
```

```
Out[... 4878705232
```

```
Out[... 4878704464
```

```
[2 3]
```

```
Out[... array([99,  3])
```

```
Out[... array([1, 2, 3, 4])
```

```
In [... # a = [1,2,3,4]
# foo = []
# for i in a:
#     if i > 2:
#         # print(i)
#         foo.append(i)

# print(foo)

# numpy way
```

```
# print(list(range(5)))
a = np.array(range(5))
a
# b = a[:5] # b is a view
b = a[a>2] # b is a copy
b
b[0]=2
b
a
```

Out[... array([0, 1, 2, 3, 4])

Out[... array([3, 4])

Out[... array([2, 4])

Out[... array([0, 1, 2, 3, 4])

mathematical operations

In [... arr = np.array([0, np.pi / 2, np.pi])

```
np.sin(arr)
np.cos(arr)
np.sqrt(arr)
import math
for i in arr:
    math.sqrt(i)
```

Out[... array([0.00000000e+00, 1.00000000e+00, 1.2246468e-16])

Out[... array([1.0000000e+00, 6.123234e-17, -1.0000000e+00])

Out[... array([0. , 1.25331414, 1.77245385])

Out[... 0.0

Out[... 1.2533141373155001

Out[... 1.7724538509055159

In [... # element-wise Operations

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
print("Addition:", arr1 + arr2)
print("Multiplication:", arr1 * arr2)
np.dot(arr1, arr2)
```

Addition: [5 7 9]
Multiplication: [4 10 18]

Out[... np.int64(32)

In [... *# Broadcasting: operations on arrays of different sha*

```
arr = np.array([1, 2, 3])  
print("Adding 5 to each element:", arr + 5)
```

Adding 5 to each element: [6 7 8]

Reshaping and Resizing

```
In [... arr = np.array([[1, 2, 3], [4, 5, 6]])  
arr  
# Reshape (2x3 to 3x2)  
reshaped_arr = arr.reshape((1,6))  
reshaped_arr  
  
arr.resize((3, 2)) # Resize (modifies the array in p  
arr
```

Out[... array([[1, 2, 3],
[4, 5, 6]])

Out[... array([1, 2, 3, 4, 5, 6])

Out[... array([[1, 2],
[3, 4],
[5, 6]])

Aggregation Functions: to compute summaries of arrays

```
In [... arr = np.array([10, 2, 3, 4, 5])  
arr.sum()  
arr.mean()  
arr.std()  
print(arr.min())  
arr.max()  
print(arr.argmin())
```

Out[... np.int64(24)

Out[... np.float64(4.8)

Out[... np.float64(2.785677655436824)

```
Out[... np.int64(10)
```

```
1
```

Vectorization vs loops

NumPy is designed to avoid explicit Python loops.

In the following example:

- `np.sin(x)` operates on the entire array at once
- This internal loop runs in compiled C, not in Python, avoiding Python loop overhead
- much faster execution for large arrays

```
In [... x = np.linspace(0, 2*np.pi, 50)
np.empty_like(x)
np.ones_like(x)
```

```
Out[... array([0.          , 0.12822827, 0.25645654, 0.3846848
1, 0.51291309,
          0.64114136, 0.76936963, 0.8975979 , 1.0258261
7, 1.15405444,
          1.28228272, 1.41051099, 1.53873926, 1.6669675
3, 1.7951958 ,
          1.92342407, 2.05165235, 2.17988062, 2.3081088
9, 2.43633716,
          2.56456543, 2.6927937 , 2.82102197, 2.9492502
5, 3.07747852,
          3.20570679, 3.33393506, 3.46216333, 3.5903916
, 3.71861988,
          3.84684815, 3.97507642, 4.10330469, 4.2315329
6, 4.35976123,
          4.48798951, 4.61621778, 4.74444605, 4.8726743
2, 5.00090259,
          5.12913086, 5.25735913, 5.38558741, 5.5138156
8, 5.64204395,
          5.77027222, 5.89850049, 6.02672876, 6.1549570
4, 6.28318531])
```

```
Out[... array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1
., 1., 1., 1., 1., 1.,
        1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1
., 1., 1., 1., 1., 1.,
        1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1
., 1., 1., 1., 1.]])
```

```
In [... x = np.linspace(0, 2*np.pi, 10000)

def loop_version(x):
    y = np.empty_like(x)
    for i in range(len(x)):
        y[i] = np.sin(x[i])
    return y

def vectorized_version(x):
    return np.sin(x)

# Run loop_version(x) `loop_num` times, measure the t

loop_num = 10

print('Loop version')
t_loop = timeit.timeit(lambda: loop_version(x), number=loop_num)
t_vector = timeit.timeit(lambda: vectorized_version(x), number=loop_num)
print(f'Loop version took {t_loop:.6f} seconds, while
```

Loop version

Loop version took 0.017060 seconds, while the vectorized version took 0.000555 seconds, making it about 30 times faster.

Introduction to Pandas

Resources:

- [Official Pandas User Guide](#)
- [Python for Data Analysis by Wes McKinney](#)

Why use Pandas?

While NumPy is excellent for numerical matrices (homogeneous data), Pandas is designed for tabular data (heterogeneous data). Think of a DataFrame as a NumPy array with labels, or an Excel sheet, allowing you to mix strings, integers, and floats in one table.

```
In [... import numpy as np
import pandas as pd
```

```
In [... # why pandas? it's useful!
foo = pd.read_html('https://arghyadutta.github.io/not
foo.to_excel('try.xlsx')
```

Pandas Series

A 1D labeled array that can hold any data type (ints, floats, strings, objects).

- Like a NumPy 1D array, but with an index
- Each value is accessed by its label, not just position
- Supports vectorized operations and alignment by index

```
In [... foo = [10, 20, 30]
foo[0]
```

```
Out[...] 10
```

```
In [... s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

# s = pd.Series({'a':10,'b':20,'c':30})
print(s)
# the values of a series are returned as numpy arrays
```

```
print(s, s.values, type(s.values))

# vectorized operation (often intuitive)
s + 5
```

```
a    10
b    20
c    30
dtype: int64
a    10
b    20
c    30
dtype: int64 [10 20 30] <class 'numpy.ndarray'>
```

```
Out[... a    15
        b    25
        c    35
        dtype: int64
```

DataFrame

2D, labeled tabular data structure.

- Rows and columns both have indexes
- Columns can hold different data types
- Supports vectorized operations, alignment, grouping, merging

The diagram illustrates a DataFrame as a 2D table. The columns are labeled 'Name', 'Team', 'Number', 'Position', 'Age', 'Height', 'Weight', 'College', and 'Salary'. The rows are indexed from 0 to 6. A 'Missing value' (NaN) is shown in the 'Number' column for row 3 and the 'Age' column for row 5. The 'Data' label points to the body of the table.

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0	6-8	235.0	LSU	1170960.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN	6-9	260.0	Ohio State	2569260.0
6	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0

Source: Wikimedia commons

```
In [... # defining a dataframe from a dictionary
data = {
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'score': [88.5, 92.0, 79.5]}
```



```

}

df = pd.DataFrame(data)
print("---- Original DataFrame ----")
print(df)

# print("the structure of your data")
# df.info()

print("get some statistical values")
print(df.describe())

print(f"\nColumns: {df.columns.tolist()}")

```

```

---- Original DataFrame ----
   name  age  score
0  Alice   25   88.5
1    Bob   30   92.0
2 Charlie   35   79.5
get some statistical values

```

	age	score
count	3.0	3.000000
mean	30.0	86.666667
std	5.0	6.448514
min	25.0	79.500000
25%	27.5	84.000000
50%	30.0	88.500000
75%	32.5	90.250000
max	35.0	92.000000

```
Columns: ['name', 'age', 'score']
```

Indexing: loc vs iloc

- `.loc` : Label-based (uses names/labels)
- `.iloc` : Integer-position based (uses 0, 1, 2...)

```

In [...] # row
df
df.loc[0]['name'] # label-based
df.loc[0,'name'] # label-based
df.iloc[0,0] # position-based

```

```
# column
df.iloc[:,1]
df["age"]
df[["name", "score"]]
```

```
Out[...      name  age  score
0    Alice   25   88.5
1     Bob   30   92.0
2  Charlie   35   79.5
```

```
Out[...  'Alice'
```

```
Out[...  'Alice'
```

```
Out[...  'Alice'
```

```
Out[...  0    25
1    30
2    35
Name: age, dtype: int64
```

```
Out[...  0    25
1    30
2    35
Name: age, dtype: int64
```

```
Out[...      name  score
0    Alice   88.5
1     Bob   92.0
2  Charlie   79.5
```

```
In [... # boolean indexing works as in numpy
df
df[df['age'] > 28]
df[df['score'] > 90]
df.query("score >= 90")

# more complex logical operation

print(df[(df['age'] > 25) & (df['score'] > 80)])
```

```
Out [...]
```

	name	age	score
0	Alice	25	88.5
1	Bob	30	92.0
2	Charlie	35	79.5

```
Out [...]
```

	name	age	score
1	Bob	30	92.0
2	Charlie	35	79.5

```
Out [...]
```

	name	age	score
1	Bob	30	92.0

```
Out [...]
```

	name	age	score
1	Bob	30	92.0

	name	age	score
1	Bob	30	92.0

Operations and column manipulations

```
In [...]
```

```
# New column
new_df = df.copy()
# id(new_df)
# id(df)
new_df['random_col'] = [1,2,3]
new_df["passed"] = new_df["score"] >= 85
new_df["old"] = new_df["age"] > 25 #why 25? nobody kn
new_df

# Vectorized (preferred)
new_df["age_squared"] = new_df["age"] ** 2

new_df.sort_values("age", ascending=False)
```

```
Out [...]
```

	name	age	score	random_col	passed	old
0	Alice	25	88.5	1	True	False
1	Bob	30	92.0	2	True	True
2	Charlie	35	79.5	3	False	True

```
Out[...      name  age  score  random_col  passed  old  age_squared
```

	name	age	score	random_col	passed	old	age_squared
2	Charlie	35	79.5	3	False	True	1225
1	Bob	30	92.0	2	True	True	900
0	Alice	25	88.5	1	True	False	625

```
In [... new_df.head(3) #first two rows
new_df.tail(2) # last two rows

# how many unique values in a particular column?
new_df.value_counts("name")
```

```
Out[...      name  age  score  random_col  passed  old  age_squared
```

	name	age	score	random_col	passed	old	age_squared
0	Alice	25	88.5	1	True	False	625
1	Bob	30	92.0	2	True	True	900
2	Charlie	35	79.5	3	False	True	1225

```
Out[...      name  age  score  random_col  passed  old  age_squared
```

	name	age	score	random_col	passed	old	age_squared
1	Bob	30	92.0	2	True	True	900
2	Charlie	35	79.5	3	False	True	1225

```
Out[... name
Alice      1
Bob        1
Charlie    1
Name: count, dtype: int64
```

Missing Data

```
In [... # # uncomment, and run. explore more!
# df_nan = df.copy()
# df_nan.loc[1, 'score'] = np.nan
# df_nan
# df_nan.isna()
# df_nan.fillna(df_nan['score'].mean())
```

GroupBy

```
In [... # df['department'] = ['CS', 'Math', 'CS']
```

```
# df
# df.groupby('department')['score'].mean()
```

Merge

```
In [... info = pd.DataFrame({
    'name': ['Charlie', 'Alice', 'Bob'],
    'city': ['Chennai', 'Delhi', 'Mumbai']
})
df
info
pd.merge(df, info, on='name')
```

```
Out [...
```

	name	age	score
0	Alice	25	88.5
1	Bob	30	92.0
2	Charlie	35	79.5

```
Out [...
```

	name	city
0	Charlie	Chennai
1	Alice	Delhi
2	Bob	Mumbai

```
Out [...
```

	name	age	score	city
0	Alice	25	88.5	Delhi
1	Bob	30	92.0	Mumbai
2	Charlie	35	79.5	Chennai

Data read and write

```
In [... df.to_csv("students.csv", index=False)
df.to_excel("students.xlsx", index=False)
```

```
In [... new_df2 = pd.read_excel('students.xlsx')
new_df2
# pd.to_csv
```

Out [...]

	name	age	score
0	Alice	25	88.5
1	Bob	30	92.0
2	Charlie	35	79.5

In [...]

```
# source: https://www.programiz.com/python-programmin

# create a dictionary containing the data
data = {'Category': ['Electronics', 'Clothing', 'Elec
          'Sales': [1000, 500, 800, 300]}

# create a DataFrame using the data dictionary
df = pd.DataFrame(data)
df
# group the DataFrame by the Category column and
# calculate the sum of Sales for each category
grouped = df.groupby('Category')['Sales'].mean()

# print the grouped data
print(grouped)
```

Out [...]

	Category	Sales
0	Electronics	1000
1	Clothing	500
2	Electronics	800
3	Clothing	300

```
Category
Clothing      400.0
Electronics   900.0
Name: Sales, dtype: float64
```

In [...]

Data visualization in Python with Matplotlib & Seaborn

- Over the years, I've collected resources on visualization, it may be useful: <https://arghyadutta.github.io/notebooks/visualization.html>
- Particularly, check this repository where I wrote some examples of how to create decent plots: <https://github.com/arghyadutta/code-i-use/tree/main/plots>
- I recommend watching [this video](#) to learn about colors and [this one](#) about layout and composition. While the videos are on graphic design, they are helpful for scientific visualization, too.
- For publication-quality plots, you need to be careful with the image size. What you see in a jupyter notebook, will **Not** look the same in a paper.
- For example, to produce a 3x2 (in square inch) figure, which is roughly the standard size for a single column and single row figure in journals, you need to use this: `fig, ax = plt.subplots(1, figsize=(3, 2))`

```
In [ ... import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns
import numpy as np
import pandas as pd

# it's generally a good idea to check the versions of

print(f"Matplotlib version: {mpl.__version__}")
print(f"Seaborn version:      {sns.__version__}")
np.__version__
```

```
Matplotlib version: 3.11.0
Seaborn version:    0.13.2
```

```
Out [ ... '2.5.1'
```

Basics

- `fig` is the whole canvas; `ax` is the individual chart area.
- `label=` and `ax.legend()` create the legend automatically.

There are many options for styling. check documentation! https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

```
In [... x1 = np.linspace(0, 2 * np.pi, 100)
y_sin = np.sin(x1)
y_cos = np.cos(x1)

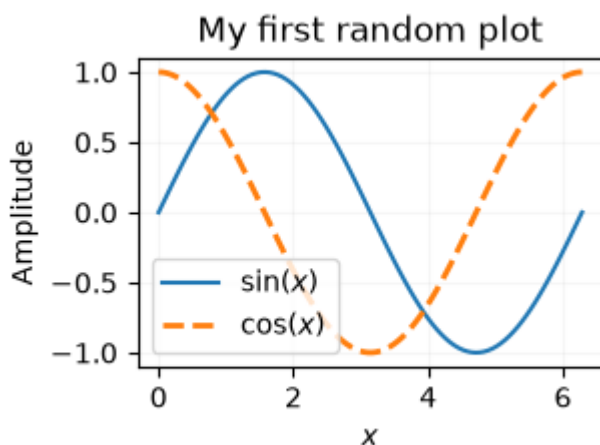
# create figure and axis
fig, ax = plt.subplots(figsize=(3,2)) # (width, height)

# plot curves

ax.plot(x1, y_sin, label=r'$\sin(x)$')

# many options to modify the plot
ax.plot(x1, y_cos, label=r'$\cos(x)$', linewidth=2, linestyle='dashed')

ax.set_title('My first random plot')
ax.set_xlabel('$x$')
ax.set_ylabel('Amplitude')
ax.legend()
ax.grid(True, alpha=0.1); # produces a light gray background
fig.savefig('p2.pdf');
```



```
In [... # sometimes for quick plots we don't use figures and
# see here for more: https://matplotlib.org/stable/tutorials/
# for example, the previous plot can be generated as:

plt.plot(x1, y_sin, label=r'$\sin(x)$', color='red')
```



```
plt.plot(x1, y_cos, label=r'$\cos(x)$', linewidth=2,

plt.title('My first random plot') # almost same comma
plt.xlabel('$x$')
plt.ylabel('Amplitude')
plt.legend()
plt.grid(True, alpha=.1); # produces a light gray bac
plt.savefig('p1.pdf')
```

Out[...]<matplotlib.lines.Line2D at 0x14fa882f0>]

Out[...]<matplotlib.lines.Line2D at 0x14fa88bc0>]

Out[... Text(0.5, 1.0, 'My first random plot')

Out[... Text(0.5, 0, '\$x\$')

Out[... Text(0, 0.5, 'Amplitude')

Out[... <matplotlib.legend.Legend at 0x14f9d9e50>



In [... # Scatter plot

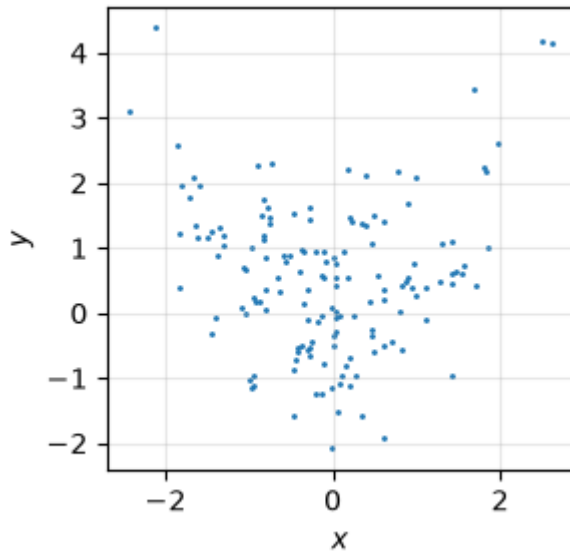
```
x = np.random.randn(150)
y = 0.6 * x**2 + np.random.randn(150)

# x = [1, 2, 3]
```

```
# y = [3, 4, 5]
fig, ax = plt.subplots(figsize=(3, 3))

ax.scatter(x, y, s=1)

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
ax.grid(True, alpha=0.3);
```



In [... *# bar plot*

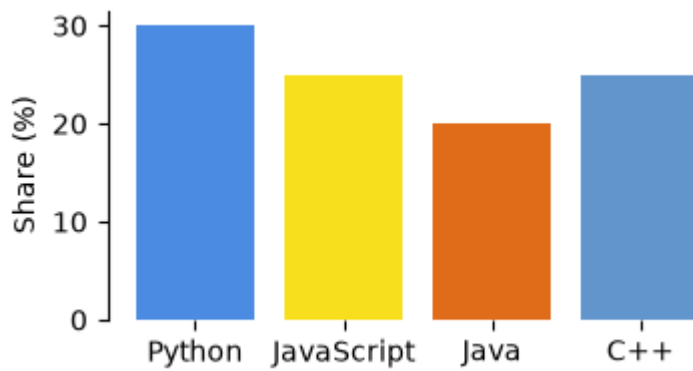
```
languages = ['Python', 'JavaScript', 'Java', 'C++']
popularity = [30, 25, 20, 25]

# you can set colors by their hex codes. look it up!
bar_colors = ['#4C8BE2', '#F7DF1E', '#E06C1A', '#6295

fig, ax = plt.subplots(figsize=(4, 2))

ax.bar(languages, popularity, color=bar_colors)
ax.set_ylabel('Share (%)')

# almost everything is editable. for example the box
ax.spines['top'].set_visible(False)    # hide top bor
ax.spines['right'].set_visible(False)  # hide right b
ax.spines['bottom'].set_visible(False);
```



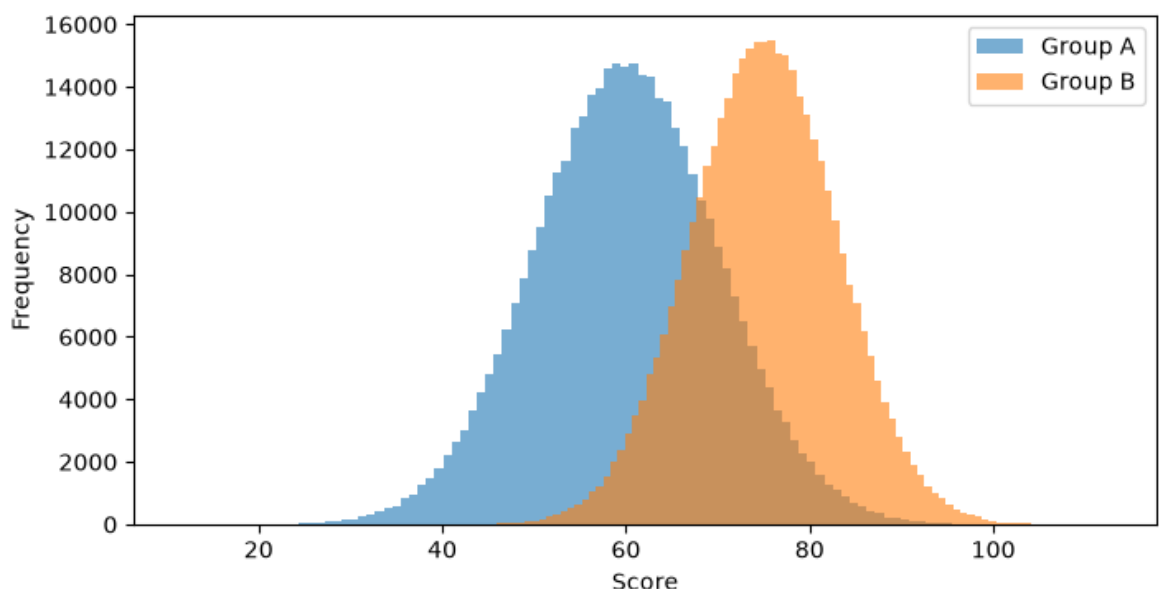
```
In [... # histogram

np.random.seed(0)
data_a = np.random.normal(loc=60, scale=10, size=4000)
data_b = np.random.normal(loc=75, scale=8, size=4000)

fig, ax = plt.subplots(figsize=(8, 4))

ax.hist(data_a, bins=100, alpha=0.6, label='Group A')
ax.hist(data_b, bins=100, alpha=0.6, label='Group B')

ax.set_xlabel('Score')
ax.set_ylabel('Frequency')
ax.legend();
```



Multiple Charts in One Figure

`plt.subplots(rows, cols)` returns a grid of axes.

```
In [... # data for plots
```

```

t = np.linspace(0, 4 * np.pi, 200)
cats = ['A', 'B', 'C', 'D']
vals = [4, 7, 3, 9]

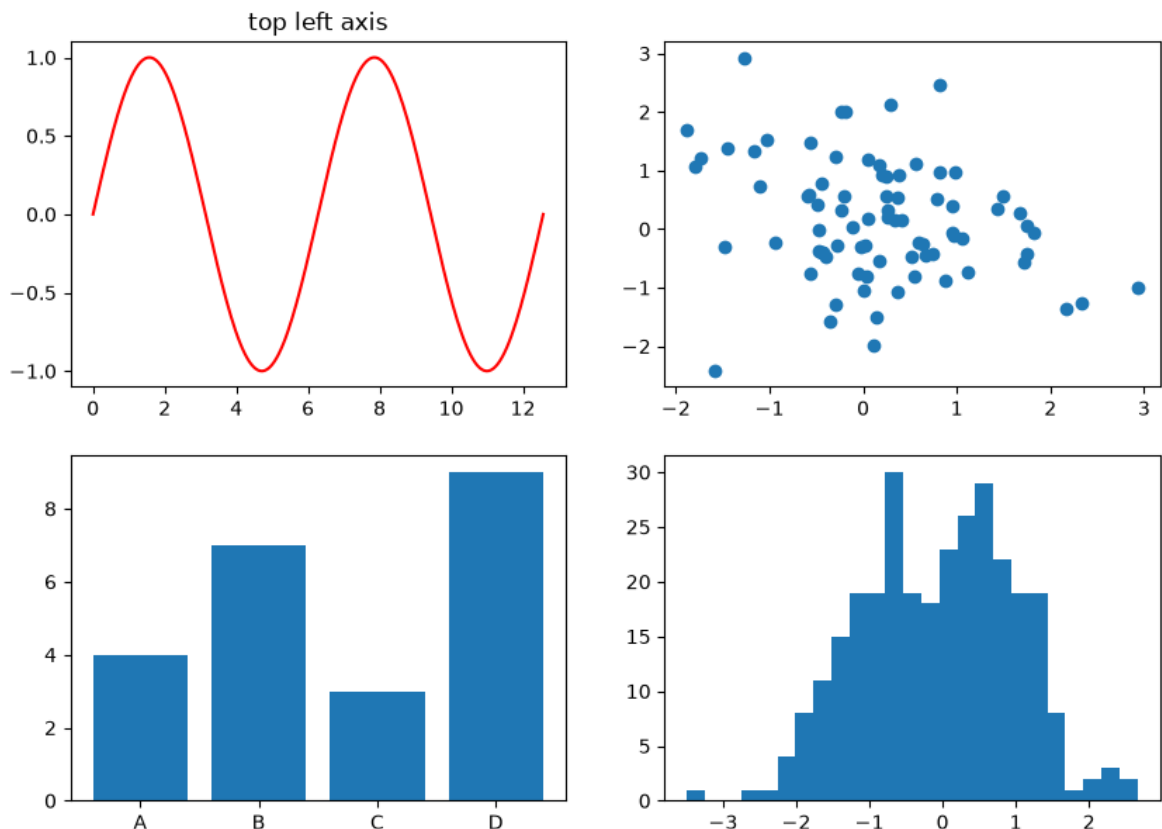
fig, axes = plt.subplots(2, 2, figsize=(10, 7))

print(type(axes))
# axes is an numpy array! and it can be indexed as us

axes[0, 0].plot(t, np.sin(t), c='red')
axes[0, 0].set_title('top left axis')
axes[0, 1].scatter(np.random.randn(80), np.random.randn(80))
axes[1, 0].bar(cats, vals)
axes[1, 1].hist(np.random.randn(300), bins=25)
fig.savefig('fourfigs.pdf');

```

```
<class 'numpy.ndarray'>
```



Seaborn

Seaborn is built on top of Matplotlib and makes statistical plots much easier. It works great with pandas DataFrames.

```

In [ ... tips = sns.load_dataset('tips') # restaurant tips d
tips.head(2)

```

```
Out [...]
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3

```
In [...]
```

```
tips.describe()
```

```
Out [...]
```

	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

```
In [...]
```

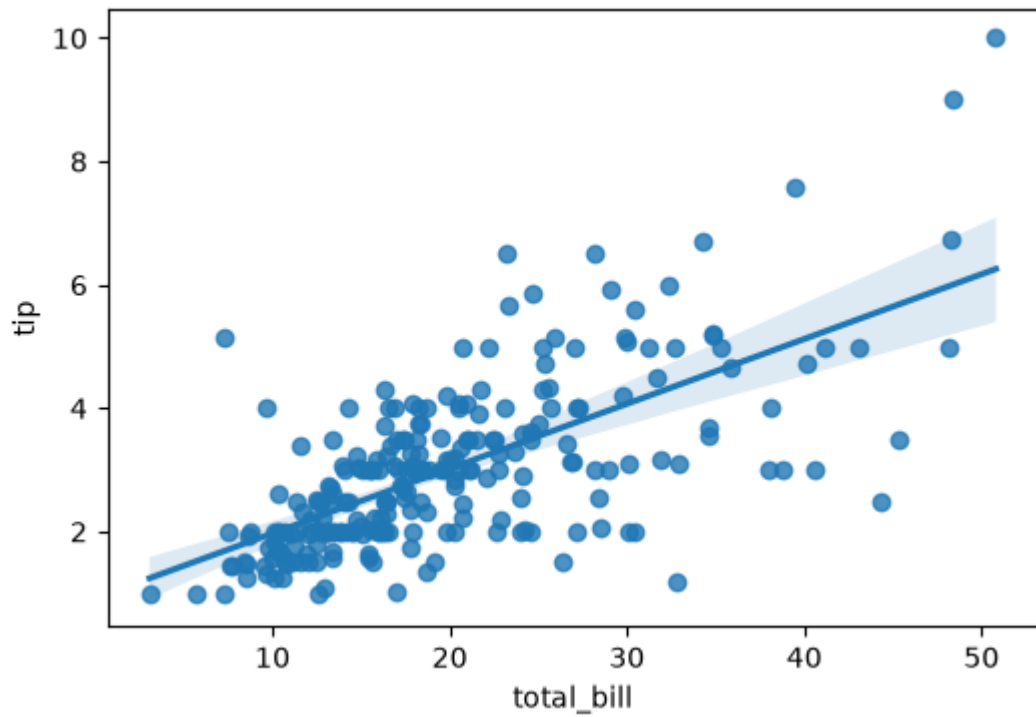
```
# regression

fig, axes = plt.subplots(1, 1, figsize=(6, 4))

sns.regplot(data=tips, x='total_bill', y='tip', ax=axes)
```

```
Out [...]
```

```
<Axes: xlabel='total_bill', ylabel='tip'>
```



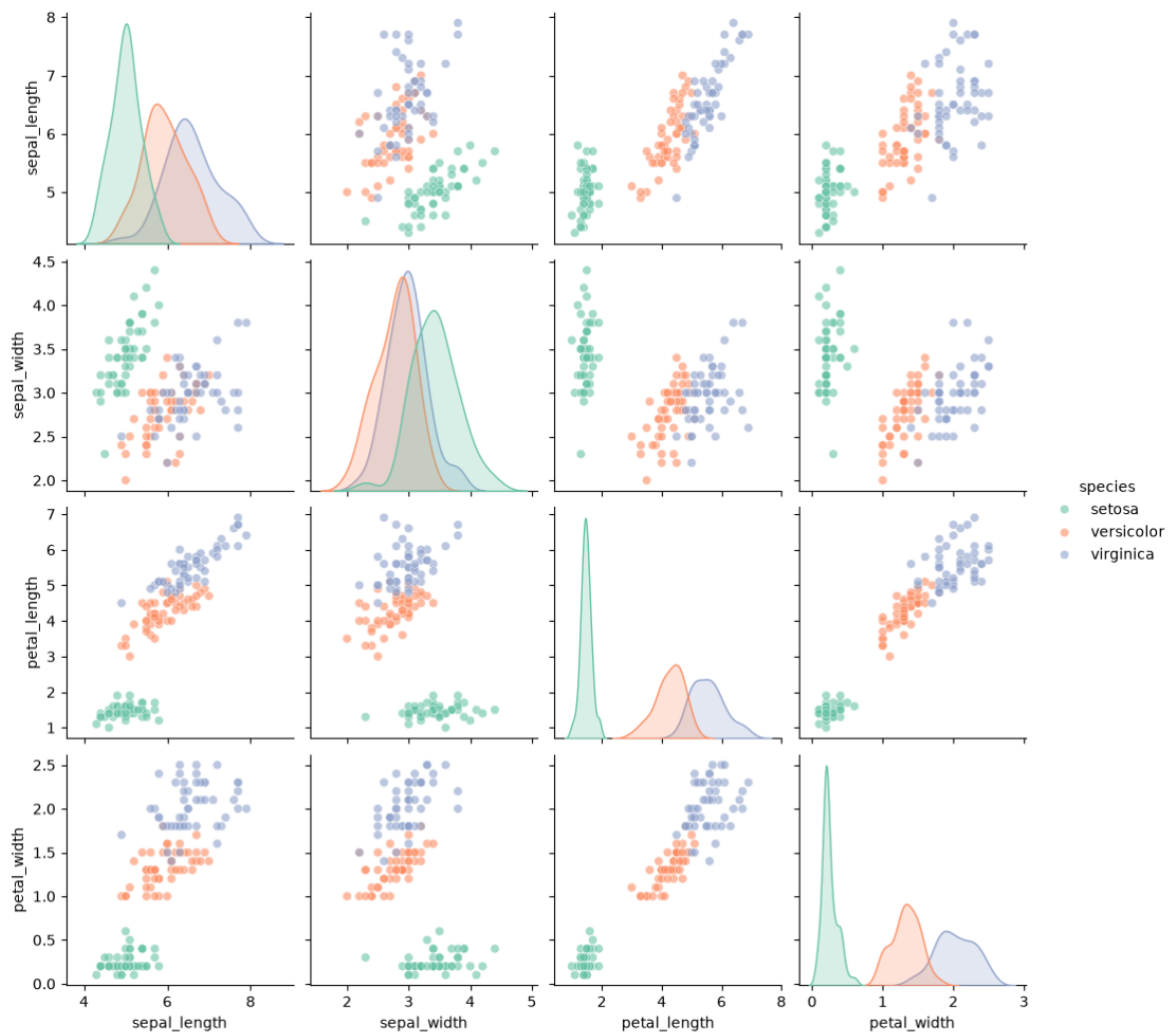
A pair plot shows all pairwise relationships in one shot. It's magical!
:)

```
In [... iris = sns.load_dataset('iris')
iris
pair_grid = sns.pairplot(iris, hue='species', palette
                        diag_kind='kde', plot_kws={'
plt.show()
```

Out [...]

	sepal_length	sepal_width	petal_length	petal_width	speci
0	5.1	3.5	1.4	0.2	setc
1	4.9	3.0	1.4	0.2	setc
2	4.7	3.2	1.3	0.2	setc
3	4.6	3.1	1.5	0.2	setc
4	5.0	3.6	1.4	0.2	setc
...	...	...	...	...	
145	6.7	3.0	5.2	2.3	virgini
146	6.3	2.5	5.0	1.9	virgini
147	6.5	3.0	5.2	2.0	virgini
148	6.2	3.4	5.4	2.3	virgini
149	5.9	3.0	5.1	1.8	virgini

150 rows × 5 columns



saving Figures

Use `fig.savefig()` to export your plots.

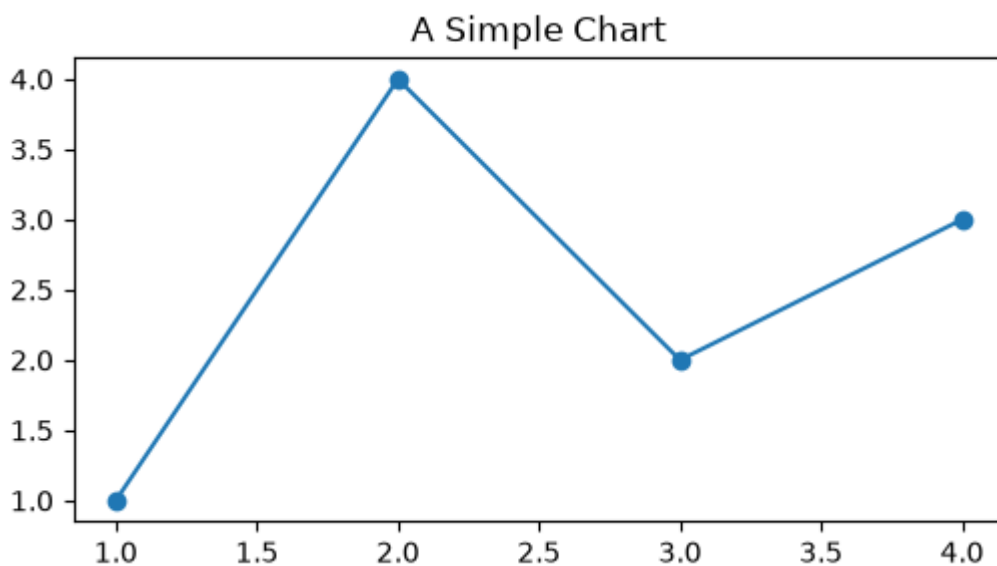
- `dpi=150` or `dpi=300` for high-quality PNGs.
- `bbox_inches='tight'` prevents labels from being cropped.
- Use `.pdf` or `.svg` for print-quality vector graphics.

```
In [... fig, ax = plt.subplots(figsize=(6, 3))
ax.plot([1, 2, 3, 4], [1, 4, 2, 3], marker='o')
ax.set_title('A Simple Chart')

# Save options
fig.savefig('my_chart.png', dpi=150, bbox_inches='tig
fig.savefig('my_chart.pdf', bbox_inches='tight')
# fig.savefig('my_chart.svg', bbox_inches='tight')
```

```
Out[... [<matplotlib.lines.Line2D at 0x160a100b0>]
```

```
Out[... Text(0.5, 1.0, 'A Simple Chart')
```



```
In [...
```



```
In [...] import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from matplotlib.animation import FuncAnimation, Pillo
```

Histograms

By partitioning the data into bins (intervals) and showing the frequency of data points that fall within each bin, a histogram shows how a numerical data is distributed. They are useful in data analysis to identify patterns or anomalies in the distribution.

From Wikipedia

"A histogram is a visual representation of the distribution of quantitative data. The term was first introduced by Karl Pearson. To construct a histogram, the first step is to "bin" (or "bucket") the range of values— divide the entire range of values into a series of intervals—and then count how many values fall into each interval. The bins are usually specified as consecutive, non-overlapping intervals of a variable. The bins (intervals) are adjacent and are typically (but not required to be) of equal size."

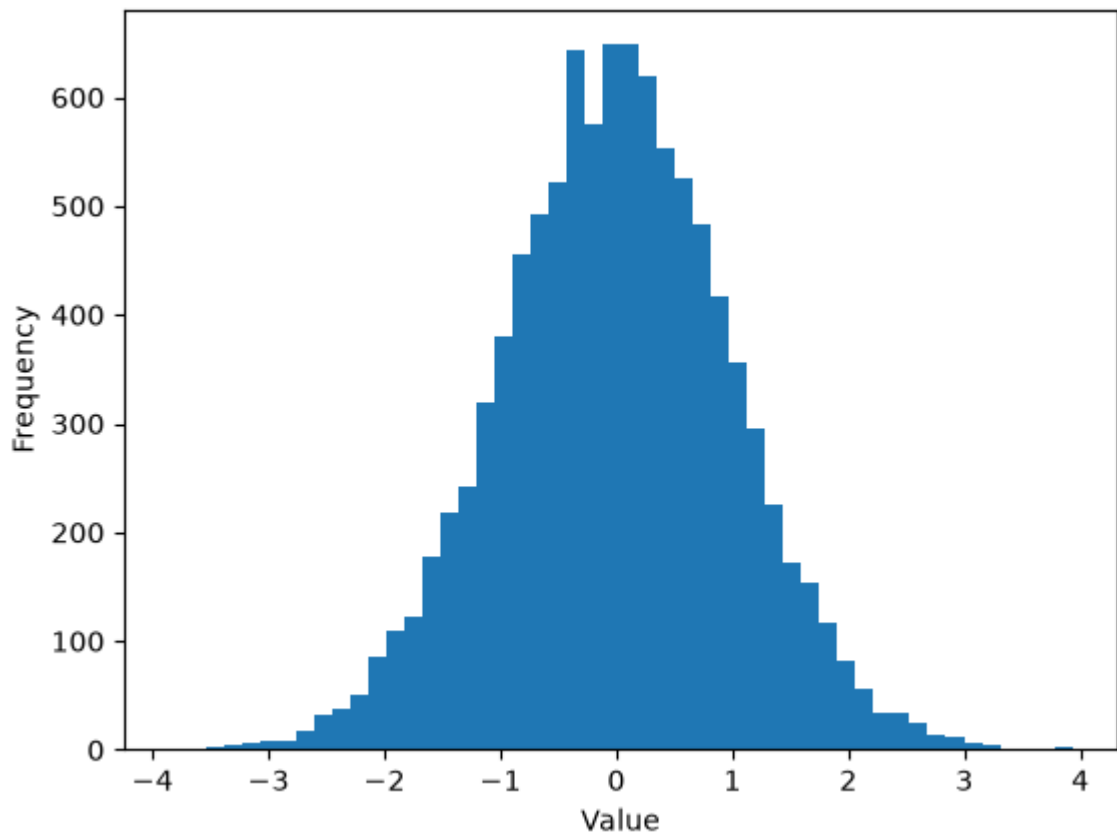
"Histograms give a rough sense of the density of the underlying distribution of the data, and often for density estimation: estimating the probability density function of the underlying variable."

```
In [...] # Creating Histograms with Matplotlib

# random data
data = np.random.randn(10000)
data
data.std()

# histogram
plt.hist(data, bins=50)
plt.xlabel("Value")
```

```
plt.ylabel("Frequency");
```

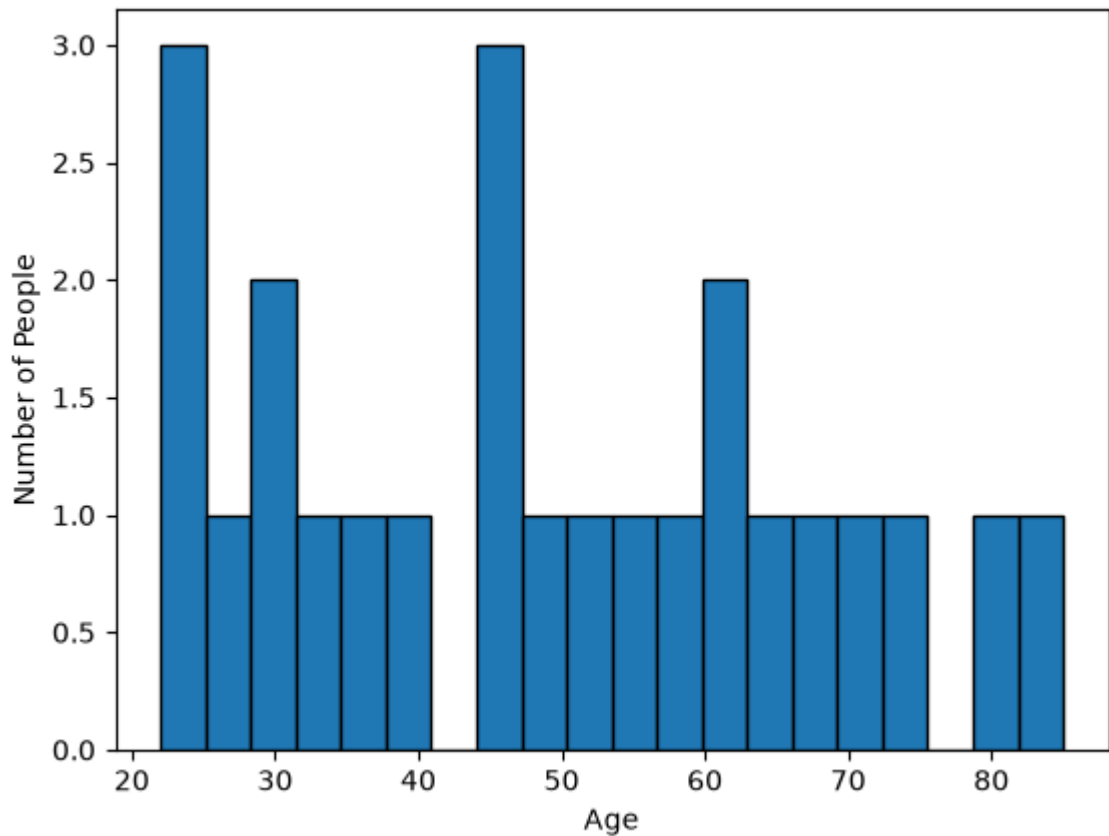


```
In [ ... # Another example: ages of people in a survey
ages = np.array([22, 23, 25, 27, 29, 30, 32, 35, 40,

len(ages)
print(ages.min(), ages.max())

plt.hist(ages, bins=20, edgecolor="black")
plt.xlabel("Age")
plt.ylabel("Number of People")
pass;
```

22 85



In [... *# check seaborn too, it's very useful for data analys*

Playing with random numbers

```
In [... import random

# random.choice(["A", 1, 1.2])

def coin_toss():
    return random.choice([1, 0])

# coin_toss()
# # Simulate N coin tosses
# for _ in range(10):
#     coin_toss()

N = 1000 # increase the number of tosses. what do yo
tosses = [coin_toss() for _ in range(N)] # list comp
# print(tosses)
count_H = 0
count_T = 0
for toss in tosses:
    if toss == 1:
```

```

        count_H = count_H + 1
    else:
        count_T = count_T + 1
print(count_H, count_T, count_H / len(tosses), count_
plt.hist(tosses)#, bins=[-0.5, 0.5, 1.5], density=True
plt.xticks([0, 1])
# plt.grid();

```

488 512 0.488 0.512

```

Out[... (array([512.,  0.,  0.,  0.,  0.,  0.,  0.,
 0.,  0., 488.]),
        array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8,
 0.9, 1. ]),
        <BarContainer object of 10 artists>)
Out[... ([<matplotlib.axis.XTick at 0x168bb34d0>,
<matplotlib.axis.XTick at 0x168b5fc20>],
[Text(0, 0, '0'), Text(1, 0, '1')])

```



```

In [... [i for i in range(3)]

```

```

Out[... [0, 1, 2]

```

```

In [... # Counting heads in two coin tosses. label 'H' by 1 a
        trials = 1000
        for i in range(1, trials + 1):
            # print(f"Number of trials = {i}")

```

```

count = 0
sum = []
for j in range(1, i + 1):
    # draw randomly from {0,1}, TWICE
    random_array = np.random.binomial(1, 0.5, size=2)
    sum_i = np.sum(random_array)
    # print(f"Outcome {j} = {random_array} and number of ones is {sum_i}")
    sum.append(sum_i)
# print(sum)
# Counter([1,2,2,3,4])
print(Counter(sum))
plt.bar(Counter(sum).keys(), Counter(sum).values(), width=0.2)
# pass;

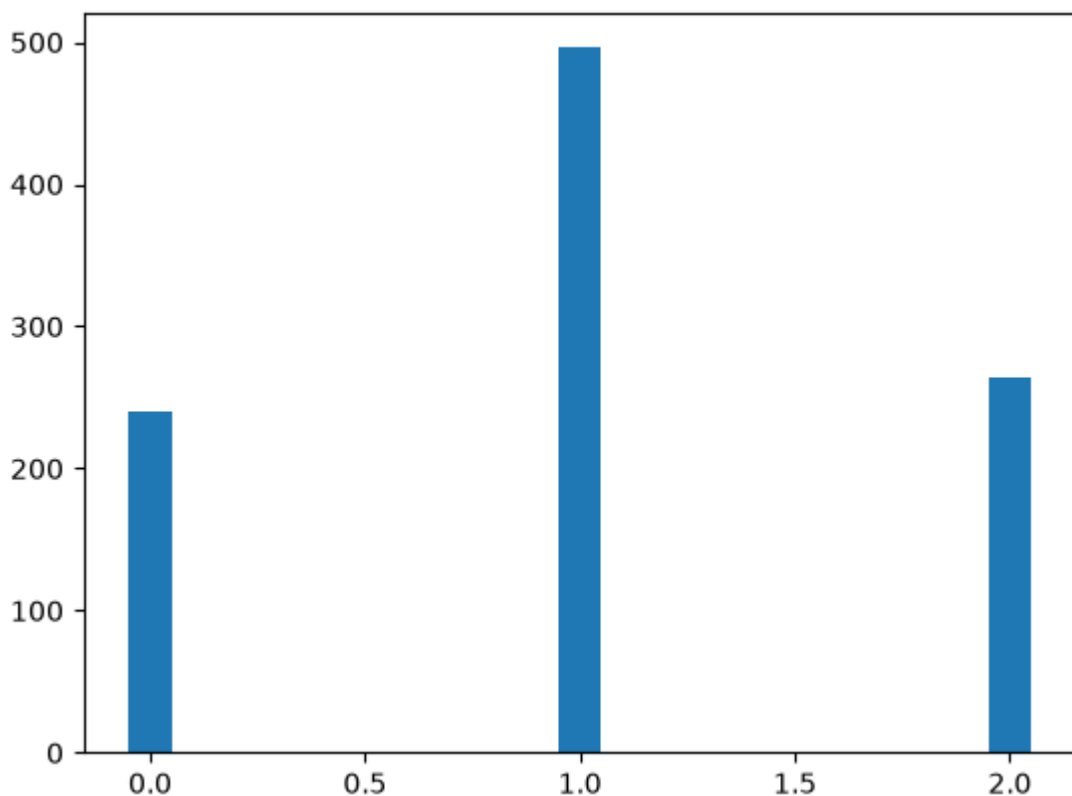
```

```

Counter({np.int64(1): 496, np.int64(2): 264, np.int64(0): 240})

```

Out[... <BarContainer object of 3 artists>



```

In [... print(np.random.choice(["H", "T"]))

```

T

```

In [... n_steps = 1000
n_walks = 100

# generate random steps of -1 or 1 for each walk

```

```

# documentation: https://numpy.org/doc/stable/referen
# IMPORTANT: Always look up documentation! Other than

steps = np.random.choice([-1, 1], size=(n_walks, n_st
# print(f"Step chosen: {steps}")
# compute cumulative sum along each row to get final
walks = np.cumsum(steps, axis=1)
# print(f"Position at each time step: {walks}")

for i in range(n_walks):
    plt.plot(walks[i], label=f"Walk {i+1}")

plt.title("Multiple 1D Random Walk Trajectories")
plt.xlabel("Steps")
plt.ylabel("Position")
pass;
# plt.legend();

```



```

In [... # Number of steps and number of walks
n_steps = 100
n_walks = 100000

steps = np.random.choice([-1, 1], size=(n_walks, n_st
walks = np.cumsum(steps, axis=1)

```

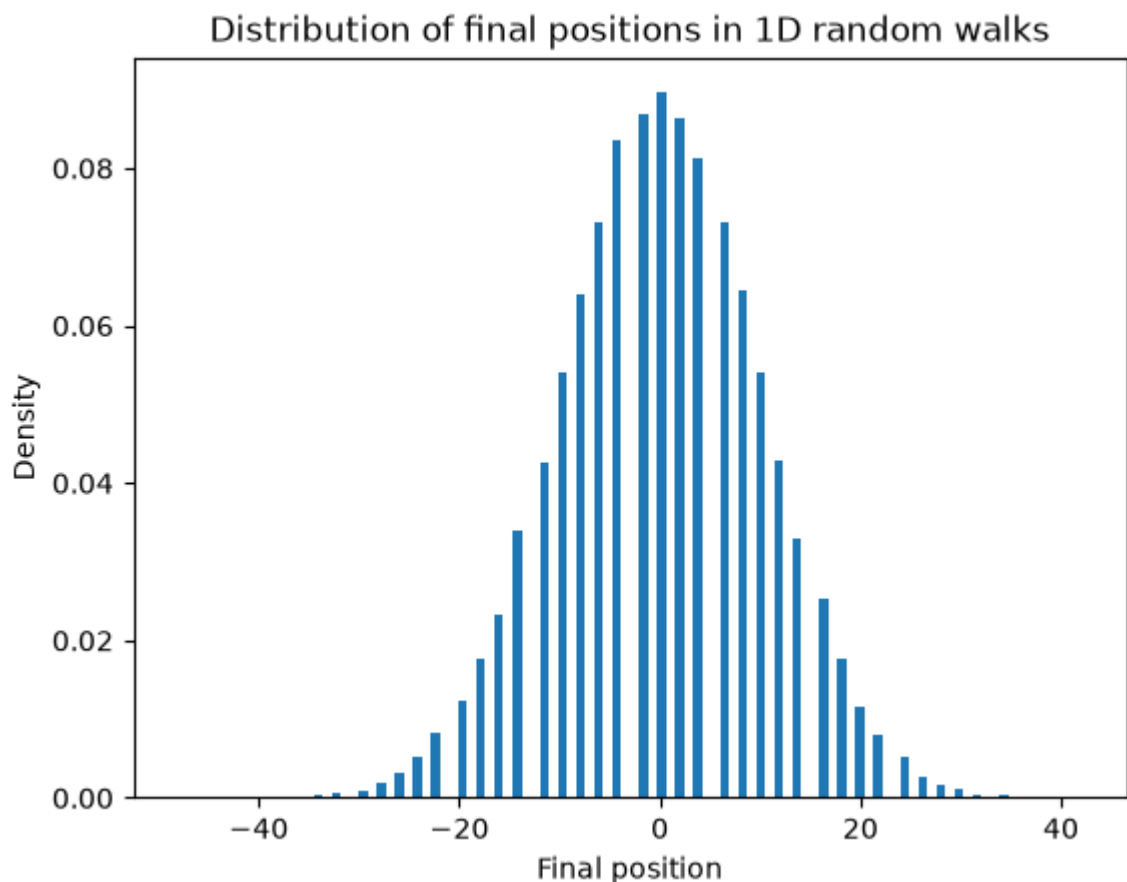
```

# print(walks)
# as I mentioned, the last element is the final position
final_positions = walks[:, -1]
# print(final_positions)

# Plot the distribution of final positions. Remember:

plt.hist(final_positions, bins=100, density=True)
plt.title("Distribution of final positions in 1D random walk")
plt.xlabel("Final position")
plt.ylabel("Density")
pass;

```



Linear regression: minimal example

```

In [ ... # install scikit learn for this. https://www.youtube.com/watch?v=...

from sklearn.linear_model import LinearRegression

X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1) # makes
# print(X)
y = np.array([3, 6, 2, 9, 11])

```

```

model = LinearRegression()
model.fit(X, y)

slope = model.coef_[0]
intercept = model.intercept_

print(f"Slope ($m$) = {slope:.2f}") # print first two
print(f"Intercept ($c$) = {intercept:.2f}")

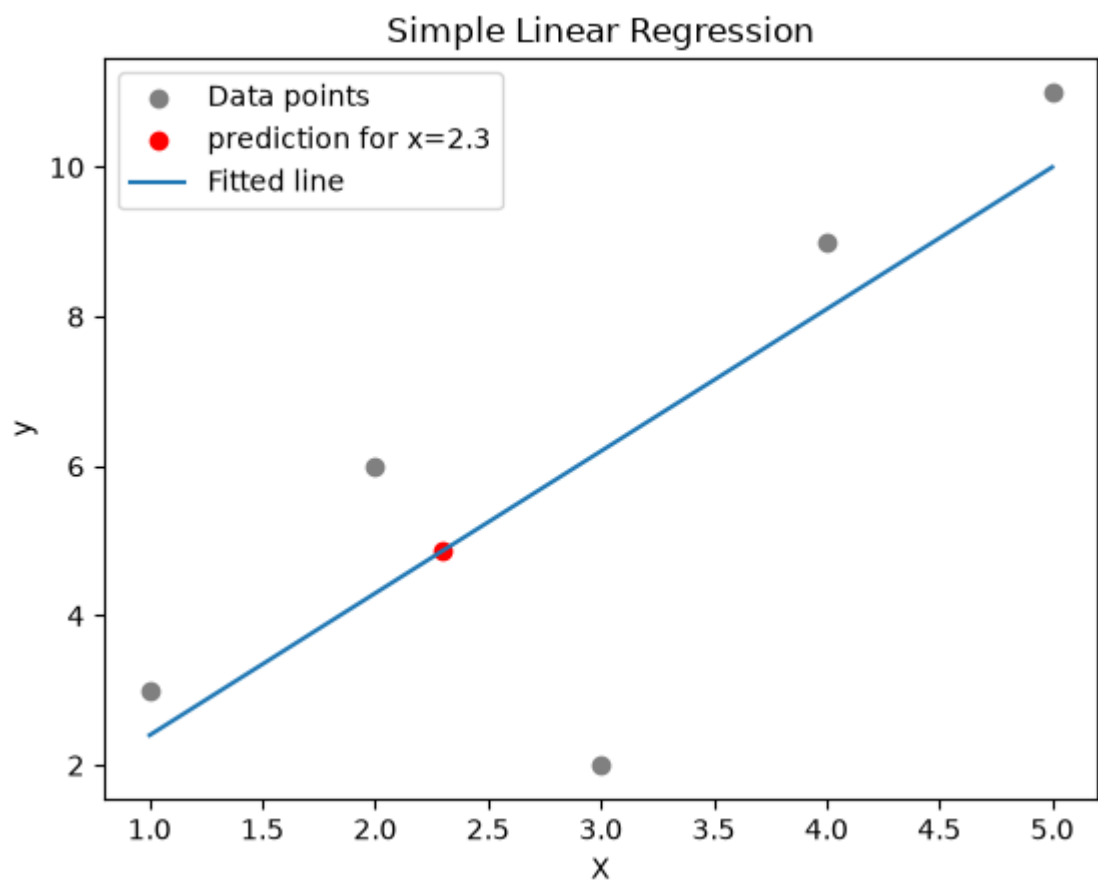
# check your fitted line for a `new` point. this is w
x_test = 2.3
X_new = np.array([[x_test]])
y_pred = model.predict(X_new)

print(f"y(x={x_test}) = {y_pred[0]:.2f}")

plt.scatter(X, y, label="Data points",color='gray')
plt.scatter(x_test,y_pred[0],color='red',label=f'pred
plt.plot(X, model.predict(X), label="Fitted line")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Simple Linear Regression")
plt.legend()
pass;

```

Slope (\$m\$) = 1.90
 Intercept (\$c\$) = 0.50
 y(x=2.3) = 4.87



In [...]